VyaparKitaab – A Cloud-Based Intelligent Business Notebook for Indian MSMEs

A Phase 1 Progress Report submitted in partial fulfilment of the requirements for the award of the degree of

Bachelor of Technology

in

Computer Science and Engineering

Specialization in

Artificial Intelligence and Machine Learning

By

Vaibhav Kumar (2415500495)

Jagriti Gupta (2415500211)

Group No. 84

Under the Guidance of

Ms. Chhavi Bajpai, Assistant Professor

Department of Computer Engineering & Applications

Institute of Engineering & Technology

GLA University

Mathura – 281406, INDIA

21 October, 2026

DECLARATION

I/We hereby declare that the work which is being presented in the B.Tech. Project **"VyaparKitaab – A Cloud-Based Intelligent Business Notebook for Indian MSMEs"**, in partial fulfilment of the requirements for the award of the **Bachelor of Technology** in Computer Science and Engineering, Specialization in Artificial Intelligence and Machine Learning, submitted to the Department of Computer Engineering and Applications of GLA University, Mathura, is an authentic record of my/our own work carried out under the supervision of **Ms. Chhavi Bajpai, Assistant Professor**.

The contents of this project report, in full or in parts, have not been submitted to any other Institute or University for the award of any degree.

Sign: _____
Name of candidate: Vaibhav Kumar
Univ Roll No.: 2415500495
Course: B.Tech CSE (AIML)

Sign: _____
Name of candidate: Jagriti Gupta
Univ Roll No.: 2415500211
Course: B.Tech CSE (AIML)

CERTIFICATE

This is to certify that the B.Tech. Project Report entitled "VyaparKitaab – A Cloud-Based Intelligent Business Notebook for MSMEs" submitted by Vaibhav Kumar Sharma (2315002323) and Yash Kumar Gaur (2315002496) in partial fulfilment of the requirements for the award of the degree of Bachelor of Technology in Computer Science and Engineering at the Department of Computer Engineering and Applications, GLA University, Mathura, is an authentic record of their own work carried out under my supervision and guidance.

The matter embodied in this project report has not been submitted earlier for the award of any degree or diploma to the best of our knowledge and belief.

Supervisor: _____

Project Co-ordinator: Dr. Mayank Srivastava

Program Co-ordinator: Dr. Nikhil Govil

Department of Computer Engineering & Applications, GLA University, Mathura (U.P.) - 281406

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to everyone who supported and guided us throughout the successful completion of this project. First and foremost, we extend our heartfelt thanks to our respected guide, **Ms. Chhavi Bajpai**, for her continuous encouragement, valuable feedback, and insightful suggestions at every stage of the project titled **"VyaparKitaab"**.

We are deeply thankful to the Department of Computer Engineering and Applications and GLA University for providing a supportive learning environment, access to resources, and the facilities that enabled the smooth execution of this project. We would also like to thank the **Head of the Department, Dr Gaurav Bathla**, for constant support and motivation, the faculty members for their guidance, and our classmates and friends for their suggestions and encouragement. Lastly, we owe our deepest appreciation to our families, whose patience and motivation helped us stay focused throughout this work.

Vaibhav Kumar (2415500495) Jagriti Gupta (2415500211)

ABSTRACT

VyaparKitaab is a cloud-based intelligent business notebook designed for Indian Micro, Small and Medium Enterprises (MSMEs). Many small shops still keep bills in paper registers, WhatsApp chats and spreadsheets, which makes billing slow, error-prone and hard to review. VyaparKitaab brings invoicing, customer and product records, supplier tracking, store and customer dashboards, OTP-verified transactions and online payment collection into one web platform.

This Phase 1 report covers the objectives completed so far, as listed in Section 1.2. The backend is built with FastAPI, asynchronous SQLAlchemy and PostgreSQL, with Alembic migrations, JWT access tokens with rotating refresh tokens, hashed one-time passwords (OTP), GSTIN, PAN and IFSC format checks, and a payment module that creates and verifies orders and handles gateway webhooks. The frontend is a React 19 and TypeScript application styled with Tailwind CSS, with separate store-owner and customer portals; it currently runs on a demo data layer while live API wiring is completed.

Infrastructure is defined in Terraform (infra/terraform) for Microsoft Azure, packaged with Docker, and checked by GitHub Actions workflows that run the backend test suite. Thirteen automated backend tests pass, and an end-to-end run against PostgreSQL 16 confirms the payment, portal and encryption flows. The Phase 2 work comprises three modules: AI/ML demand forecasting and cash-flow analytics, live SMS and WhatsApp gateway delivery, and PostgreSQL row-level security, together with Redis-backed OTP storage and live API integration of every screen.

TABLE OF CONTENTS

CERTIFICATE ii

DECLARATION ..... **ii**
ACKNOWLEDGEMENT ..... **iii**
ABSTRACT ..... **iv**
CHAPTER 1: INTRODUCTION **1**
1.1 Overview and Motivation 1
1.2 Objectives 1
1.3 Summary of Similar Applications 2
1.4 Organization of the Project Report 3
1.5 Scope and Boundaries 3
CHAPTER 2: SOFTWARE REQUIREMENT ANALYSIS **5**
2.1 Introduction to Requirement Analysis 5
2.2 Feasibility Study 5
2.3 Functional Requirements 5
2.4 Non-Functional Requirements 6
2.5 Use Case Descriptions 7
2.6 Module Descriptions 8
2.7 Hardware and Software Requirements 9
CHAPTER 3: SOFTWARE DESIGN **10**
3.1 System Architecture Overview 10
3.2 Data Flow Diagrams 10
3.3 UML Diagrams 11
3.4 Database Design 12
3.5 Security Design 15
3.6 Authentication and Session Flow 16
3.7 Deployment Architecture 17
3.8 API Design Conventions 18
CHAPTER 4: IMPLEMENTATION AND USER INTERFACE **19**
4.1 Technology Stack Used 19
4.2 Backend Implementation 19
4.3 AI/ML Demand Forecasting, Personal Purchase Vault and Row-Level Security
4.4 Frontend Implementation 20
4.5 Configuration and Environment Variables 25
4.6 Authentication Implementation 26
4.7 KYC Format Validation 27
4.8 Payment Module Implementation 27
4.9 Continuous Integration and Delivery 28
4.10 Screens and User Walkthrough 28
CHAPTER 5: SOFTWARE TESTING **30**
5.1 Unit Testing 30
5.2 Black Box Testing 31
5.3 White Box Testing 32

5.4 Testing Status and Pending Tests....32
5.5 End-to-End Smoke Testing....32
5.6 Reproducing the Test Run....33
5.7 Discussion of Test Results....34
CHAPTER 6: CONCLUSION....35
6.1 Limitations of the Phase 1 Build....35
6.2 Recommended Next Steps....35
CHAPTER 7: SUMMARY....37
REFERENCES....38
APPENDIX A: ABBREVIATIONS....40
APPENDIX B: SAMPLE API ENDPOINTS....41
APPENDIX C: SOURCE CODE EXCERPTS....42
APPENDIX D: DATABASE MIGRATION HISTORY....43
APPENDIX E: FRONTEND PAGE MAP....44
APPENDIX F: SEED PRODUCT CATALOGUE....45
APPENDIX G: SEED CUSTOMER RECORDS....47

LIST OF TABLES

Table 1.1: Comparison of similar applications with VyaparKitaab....3
Table 2.1: Feasibility study summary....5
Table 2.2: Functional requirements of VyaparKitaab and their status....6
Table 2.3: Non-functional requirements of VyaparKitaab....7
Table 2.4: Use case: store owner creates an invoice....7
Table 2.5: Use case: customer pays an invoice online....7
Table 2.6: Use case: customer signs in with a mobile OTP....8
Table 2.7: Development and demonstration environment....9
Table 3.1: Database table: users....13
Table 3.2: Database table: businesses....13
Table 3.3: Database table: customers....13
Table 3.4: Database table: products....14
Table 3.5: Database table: suppliers....14
Table 3.6: Database table: invoices....14
Table 3.7: Database table: invoice_items....14
Table 3.8: Database table: otp_verifications....15
Table 3.9: Database table: refresh_tokens....15
Table 3.10: Security controls and where they are implemented....16
Table 3.11: Lifetimes of tokens and codes....17
Table 3.12: Azure resources declared in infra/terraform....18
Table 3.13: HTTP status codes used by the API....18
Table 4.1: Technology stack summary....19
Table 4.2: Backend API modules....20
Table 4.3: Frontend routes and their purpose....21
Table 4.4: Backend configuration settings....26
Table 4.5: Format rules applied by the KYC module....27
Table 4.6: GitHub Actions workflows....28
Table 5.1: Unit testing results – backend....30
Table 5.2: Black box test cases – authentication, invoices and payments....31
Table 5.3: White box review – payment module....32
Table 5.4: Summary of testing status at Phase 1....32
Table 5.5: End-to-end checks against PostgreSQL 16....33
Table A.1: List of abbreviations....40
Table B.1: Endpoints of the VyaparKitaab backend (all under /api/v1)....41
Table D.1: Database migration history....43
Table E.1: Frontend page components, routes and purposes....44
Table F.1: Seed product catalogue (30 products)....46
Table G.1: Seed customer records....47

LIST OF FIGURES

Figure 1.1: Business workflow supported by VyaparKitaab in Phase 1 4
Figure 3.1: System architecture of VyaparKitaab 10
Figure 3.2: Data flow diagrams: Level 0 (a) and Level 1 (b) 11
Figure 3.3: Entity–relationship diagram of the VyaparKitaab database 15
Figure 4.1: Landing page 22
Figure 4.2: Login and registration screens 22
Figure 4.3: Store owner dashboard 23
Figure 4.4: Customer and product management screens 23
Figure 4.5: Invoice creation screen 24
Figure 4.6: Invoice list and invoice detail 24
Figure 4.7: Customer portal: bills and spending 25
Figure 4.8: Interactive API documentation 25
Figure 4.9: Store-owner and customer journeys through the Phase 1 build 29
Figure 5.1: Unit testing output – backend 31
Figure 6.1: Phase 1 foundation and the planned Phase 2 work 36

CHAPTER 1: INTRODUCTION

1.1 Overview and Motivation

Small retail shops, wholesalers and service businesses form the backbone of the Indian economy, yet many of them still run on paper bill books, WhatsApp messages and loose spreadsheets. Creating a bill by hand takes time, customer records get lost, GST details are calculated manually, and the owner has little visibility into how the business is performing. Customers, in turn, have no simple way to see what they bought, what they paid and how much they have spent at a store.

Existing billing tools solve parts of this problem, but they are often built for the business owner alone. Customers are not given a secure view of their own transactions, and the link between a bill, a payment and a customer record is usually weak. VyaparKitaab is proposed to close this gap. It gives the store owner a single notebook for billing, customers, products and suppliers, and gives each customer a separate, protected portal that shows only that customer's own bills and spending.

The project is also motivated by the need to demonstrate sound engineering practice: an asynchronous API, a relational data model with migrations, token-based authentication, payment integration, containerised deployment and infrastructure written as code. The work therefore serves both as a practical tool for small businesses and as a complete software-engineering exercise.

1.2 Objectives

The objectives of the project are divided into two groups. The first group has been completed in Phase 1. The second group is planned for the next phase.

Completed Objectives (Phase 1)

1. To build a secure authentication system with registration, login, one-time password (OTP) verification, rotating refresh tokens and logout.
2. To provide business profile management with GSTIN, address and contact details for each store.
3. To manage customers, products (with SKU and stock) and suppliers through a business-scoped API.

4. To create GST-aware invoices with line items, discounts and automatic totals, and to track each invoice through DRAFT, CONFIRMED, PAID and CANCELLED states.
5. To provide store-owner and customer dashboards that summarise sales and spending.
6. To create and verify online payment orders for invoices, with signature checking and a webhook endpoint for asynchronous gateway events.
7. To validate Indian business identifiers (GSTIN, PAN, IFSC) and bank account formats before records are saved.
8. To build a routed React frontend with separate store-owner and customer portals.
9. To containerise the backend, define the Azure infrastructure in Terraform, and run automated checks through GitHub Actions.
10. To encrypt personal data at rest with field-level Fernet encryption and keyed blind indexes for lookups, and to store each payment order on its invoice.

Pending Objectives (Phase 2)

- Enforce row-level security in PostgreSQL and use a least-privilege application database role.
- Move OTP storage to Redis with time-to-live expiry, and add rate limiting and security headers to the API.
- Connect every frontend screen to the live API in place of the demo data layer.
- Deliver OTPs and payment receipts through a live SMS and WhatsApp gateway; OTP delivery is currently a logging stub.
- Build the AI/ML demand forecasting and cash-flow analytics module.
- Extend automated testing to integration, end-to-end and load tests.

1.3 Summary of Similar Applications

Several established platforms already serve Indian small businesses. Table 1.1 compares the most widely used of them with VyaparKitaab on the dimensions that matter for this project. The comparison is based on the public product pages of each platform and is intended to show positioning, not to rank products.

Application	Key features	Customer-facing view	Limitations relevant to this project
Vyapar	GST billing, inventory, accounting, reports; mobile and desktop	Not a separate portal	Owner-centric; customer side limited to receipts

Application	Key features	Customer-facing view	Limitations relevant to this project
myBillBook	GST invoices, e-way bills, inventory, payment reminders, ledger sharing	Ledger sharing with parties	Designed around the business owner's books
Khata Billing	Free GST invoicing, inventory and basic analysis	Not evident from public listing	Simpler feature set; limited analytics
Swipe	Sales and purchase invoices, stock, payment links and QR codes	Payment link to customer	Payment-focused; no dual-role data separation
VyaparKitaab (this project)	Invoicing, customers, products, suppliers, store and customer dashboards, OTP verification, online payment with webhook	Dedicated customer portal showing only own bills and spending	Phase 1: frontend on demo data; AI features pending

Table 1.1: Comparison of similar applications with VyaparKitaab

1.4 Organization of the Project Report

This report is organised into seven chapters. Chapter 1 introduces the problem, objectives and similar applications. Chapter 2 presents the feasibility study, functional and non-functional requirements, use cases and module descriptions. Chapter 3 describes the system design, including architecture, data flow, UML views and the database design. Chapter 4 documents the implementation, the technology stack, the API and the user interface. Chapter 5 reports the testing carried out so far and the testing still to be done. Chapter 6 concludes the Phase 1 work, and Chapter 7 gives a summary. The references and appendices follow.

1.5 Scope and Boundaries

The project covers the everyday billing workflow of a small shop: records of customers, products and suppliers, GST-aware invoices, dashboards, and online payment collection for an invoice. The customer side is limited to viewing the customer's own bills and spending, and paying an outstanding invoice. The project does not cover accounting ledgers, inventory valuation, e-way bills or income-tax filing, and it does not replace the statutory records that a business must keep.

Three boundaries shape the design. First, business and banking identifiers are checked for format only. Second, payments are made through the gateway's own checkout, and the application stores the payment method and the order reference but no card details.

Third, Phase 1 delivers a working backend and a frontend that demonstrates the flows; the Phase 2 work listed in Section 4.3 is outside the scope of this report.

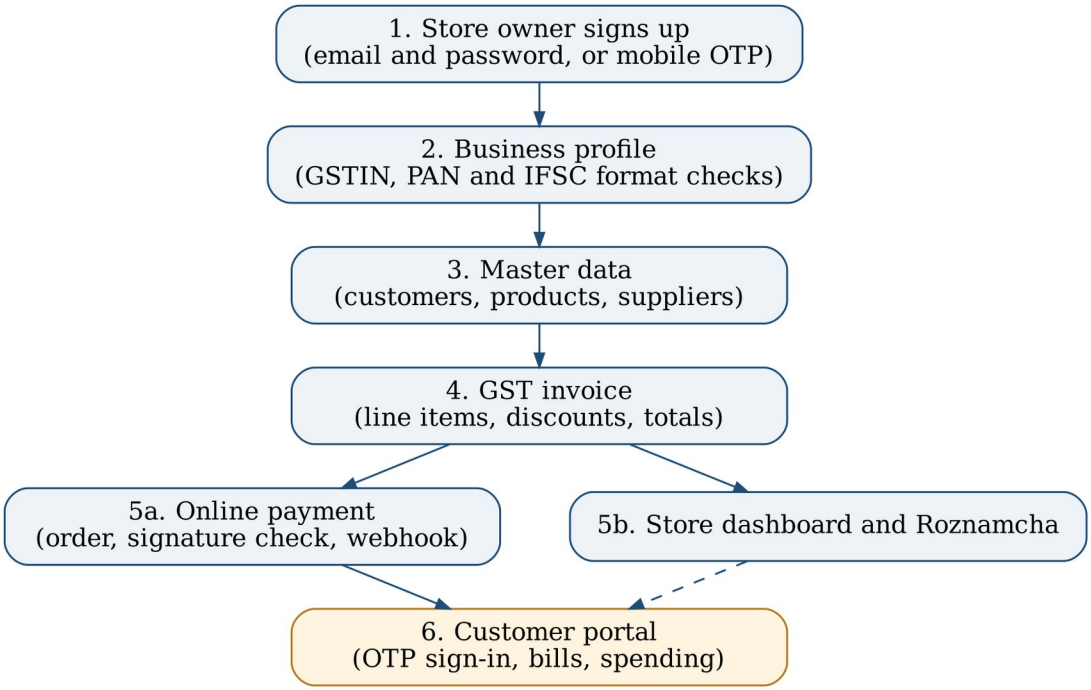


Figure 1.1: Business workflow supported by VyaparKitaab in Phase 1

CHAPTER 2: SOFTWARE REQUIREMENT ANALYSIS

2.1 Introduction to Requirement Analysis

Requirement analysis identifies what the system must do, how well it must do it, and how users will interact with it. For VyaparKitaab the primary users are the **store owner** (who runs billing, records and reports), the **customer** (who views bills and pays online) and **staff** (a role reserved for future use). The requirements below were derived from the problem described in Chapter 1 and from the features of existing billing tools.

2.2 Feasibility Study

The feasibility of the project was assessed across technical, operational, economic, legal and schedule dimensions. The results are summarised in Table 2.1.

Feasibility type	Description	Analysis	Conclusion
Technical	Availability of frameworks, database, cloud and payment services	FastAPI, React, PostgreSQL, Docker and Azure services are mature and well documented. The payment module supports a sandbox mode for development.	Feasible
Operational	Ease of use for owners and customers with little accounting background	Simple invoice form and card-based dashboards. Separate customer portal avoids exposing owner data.	Feasible
Economic	Development and running cost for a small business	Open-source stack. Azure Container Apps can scale to zero, keeping idle cost low.	Cost-effective
Legal	Compliance with data-protection and tax requirements	Hashed passwords and OTPs, JWT authentication and business-scoped queries are in place. Personal data is encrypted at rest, supporting obligations under the Digital Personal Data Protection Act, 2023.	Feasible
Schedule	Completion within the academic semester	Work is split into sprints. Core features are complete for Phase 1; AI features are planned for Phase 2.	Feasible

Table 2.1: Feasibility study summary

2.3 Functional Requirements

Functional requirements describe the behaviours the system must support. Table 2.2 lists the major requirements together with their status at the end of Phase 1.

ID	Requirement	Status
FR-01	A store owner can register an account and log in with email and password	Completed
FR-02	Access tokens are issued with rotating refresh tokens; the user can log out	Completed
FR-03	OTP is generated, stored as a hash, expires, and can be used only once	Completed (database storage)
FR-04	A customer can log in with a phone-based OTP	Completed
FR-05	A business profile with GSTIN, address and phone can be created and listed	Completed
FR-06	Customers can be created and listed for a business	Complete
FR-07	Products with SKU, unit, prices and stock can be created and listed	Complete
FR-08	Suppliers can be created and listed for a business	Completed
FR-09	An invoice with items, discounts and GST is created with an automatic invoice number and totals	Completed
FR-10	Invoices can be listed, opened, and moved between statuses	Completed
FR-11	Store-owner and customer dashboards show totals and spending	Completed (backend); frontend on demo data
FR-12	A payment order can be created for an invoice, verified, and reconciled by webhook	Completed (sandbox; order stored on the invoice)
FR-13	GSTIN, PAN, IFSC and bank account numbers are format-checked before use	Partial (format checks only)
FR-14	AI/ML demand forecasting and cash-flow analytics	Pending (Phase 2)
FR-15	OTPs and payment receipts are delivered by live SMS and WhatsApp	Pending (Phase 2); OTP messages are logged only

Table 2.2: Functional requirements of VyaparKitaab and their status

2.4 Non-Functional Requirements

Non-functional requirements define the quality attributes of the system. Table 2.3 summarises them and the current status of each.

Attribute	Requirement	Current status
Security	Passwords hashed; tokens signed; data isolated per business	Passwords use bcrypt with 12 rounds; JWT access tokens; business-scoped queries; personal data encrypted at rest. Row-level security pending.
Privacy	Personal data protected at rest	Fernet (AES-based) encryption applied to business and customer contact fields; keyed blind indexes support lookups.
Performance	Responsive under concurrent requests	Asynchronous FastAPI with asyncpg; load testing pending.
Availability	Service recovers from restarts and scales with demand	Health endpoints (/api/v1/health and /api/v1/health/db) present; Azure Container Apps defined in Terraform.

Attribute	Requirement	Current status
Usability	Usable on desktop and mobile screens	Responsive Tailwind layouts; customer portal designed for phones.
Maintainability	Modular code with migrations and automated checks	Routers, services, schemas and models separated; Alembic migrations; pytest and CI.
Portability	Runs the same locally and in the cloud	Docker and Docker Compose for local runs (PostgreSQL 15); Terraform in infra/terraform for Azure; Render Blueprint (render.yaml) also provided.

Table 2.3: Non-functional requirements of VyaparKitaab

2.5 Use Case Descriptions

Two principal use cases are described below. Table 2.4 covers the store owner creating an invoice, and Table 2.5 covers a customer paying an invoice online.

Element	Description
Actor	Store owner
Precondition	Owner is logged in; a business profile exists; products exist in the catalogue
Main flow	1. Owner selects or creates a customer. 2. Owner adds items with quantity and discount. 3. System computes subtotal, GST and total. 4. Invoice is saved as DRAFT with a generated invoice number. 5. Owner confirms the invoice, moving it to CONFIRMED.
Alternate flow	The OTP endpoints exist for a future confirmation check; this check is not part of the Phase 1 flow.
Exception	Missing product or invalid quantity returns a validation error; the invoice is not saved.
Postcondition	Invoice is stored and appears in the invoice list and the dashboard totals.

Table 2.4: Use case: store owner creates an invoice

Element	Description
Actor	Customer (logged in through the OTP login)
Precondition	Invoice exists, belongs to the customer, and is not fully paid
Main flow	1. Customer opens the bills list in the portal. 2. Customer opens an invoice. 3. System creates a payment order for the outstanding amount. 4. Customer pays through the gateway checkout. 5. System verifies the payment signature and marks the invoice PAID. 6. Webhook events reconcile late or repeated notifications.
Exception	Invoice already fully paid returns HTTP 400. Invoice not owned by the requester returns HTTP 404.
Postcondition	Invoice status and paid amount are updated; the customer dashboard reflects the payment.

Table 2.5: Use case: customer pays an invoice online

2.6 Module Descriptions

The system is divided into ten modules, each owning a clear part of the functionality.

- **Authentication and OTP:** registration, login, token refresh and logout; OTP generation, hashing, verification and customer OTP login.
- **Business and master data:** business profile, customers, products and suppliers, all scoped to the owner's business.
- **Transactions:** invoice creation with items and GST, invoice listing and detail, and status transitions.
- **Dashboards and spending:** store totals and customer spending summaries computed from invoices.
- **Payments:** order creation, signature verification, payment information lookup and the gateway webhook.
- **KYC verification:** format validation for GSTIN, PAN, IFSC and bank account numbers.
- **Security utilities:** password hashing, JWT creation and decoding, role checks, and field-level encryption with blind indexes.
- **Store portal (frontend):** dashboard, customers, products, invoice list, invoice creation and invoice detail.
- **Customer portal (frontend):** bills, bill detail and spending dashboard, separated from the owner's views.
- **Infrastructure and delivery:** Docker, Docker Compose, Terraform in infra/terraform, and GitHub Actions workflows.

Element	Description
Actor	Customer
Precondition	The customer's mobile number matches a customer record or a bill issued to that number
Main flow	1. Customer enters the mobile number and requests a code. 2. System stores a hash of a one-time code with an expiry of five minutes. 3. Customer enters the code. 4. System checks the code against the stored hash, marks it used and issues an access token and a refresh token. 5. The portal shows the customer's own bills.
Exception	A malformed number is refused with HTTP 400. A wrong or expired code is rejected, and the sandbox code is not accepted outside development and testing.
Postcondition	The customer holds an authenticated session limited to the customer's own invoices.

Table 2.6: Use case: customer signs in with a mobile OTP

2.7 Hardware and Software Requirements

Table 2.7 lists the environment needed to develop and demonstrate the project. The production target is a managed container platform with a managed PostgreSQL service, as described in Section 3.7.

Item	Requirement	Reason
Operating system	A 64-bit system able to run Docker, or Linux or Windows with WSL2	Runs the containers and the local tools
Memory	8 GB RAM recommended	Runs PostgreSQL, the backend, the frontend development server and a browser together
Backend runtime	Python 3.12	Version used by the Dockerfile and the backend CI job
Frontend runtime	Node.js 20	Version used by the frontend container in Docker Compose
Database	PostgreSQL 15 with Docker Compose; PostgreSQL 16 for the test run in Chapter 5	Relational storage, accessed through the asyncpg driver
Containers	Docker Engine with Docker Compose	Local packaging and the deployed image
Browser	A current version of Chrome, Edge or Firefox	Runs the single-page application

Table 2.7: Development and demonstration environment

CHAPTER 3: SOFTWARE DESIGN

3.1 System Architecture Overview

VyaparKitaab follows a client–server architecture. The React single-page application runs in the browser and calls a versioned REST API under **/api/v1**. The FastAPI backend validates requests with Pydantic schemas, applies business rules in service modules, and stores data in PostgreSQL through asynchronous SQLAlchemy. Refresh tokens are stored only as SHA-256 hashes and are rotated when used. The payment gateway is reached from the backend only, and it calls back into the webhook endpoint. Figure 3.1 shows the layers of the system.

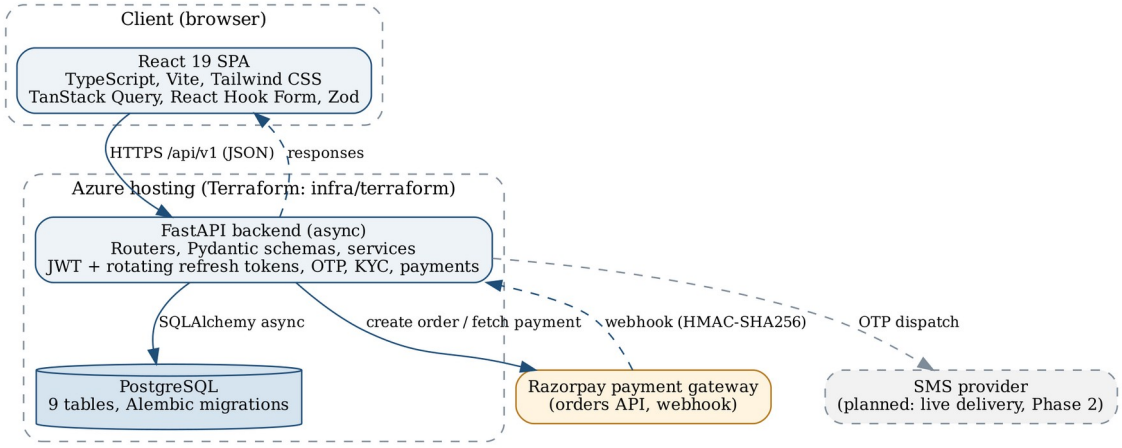


Figure 3.1: System architecture of VyaparKitaab

3.2 Data Flow Diagrams

The Level 0 data flow diagram (Figure 3.2a) shows the system as a single process exchanging data with four external entities: the store owner, the customer, the payment gateway and the SMS service used for OTP delivery. The owner supplies business, customer, product and invoice data and receives dashboards and invoice details. The customer supplies OTP responses and payment confirmations and receives bills and spending summaries. The payment gateway returns order details and sends webhook events. In Phase 1 the SMS service is a logging stub; live delivery is planned for Phase 2. The Level 1 diagram (Figure 3.2b) decomposes the system into six processes (authentication and OTP, business and master data, invoicing and GST, KYC verification, payments, and dashboards and spending) and three data stores.

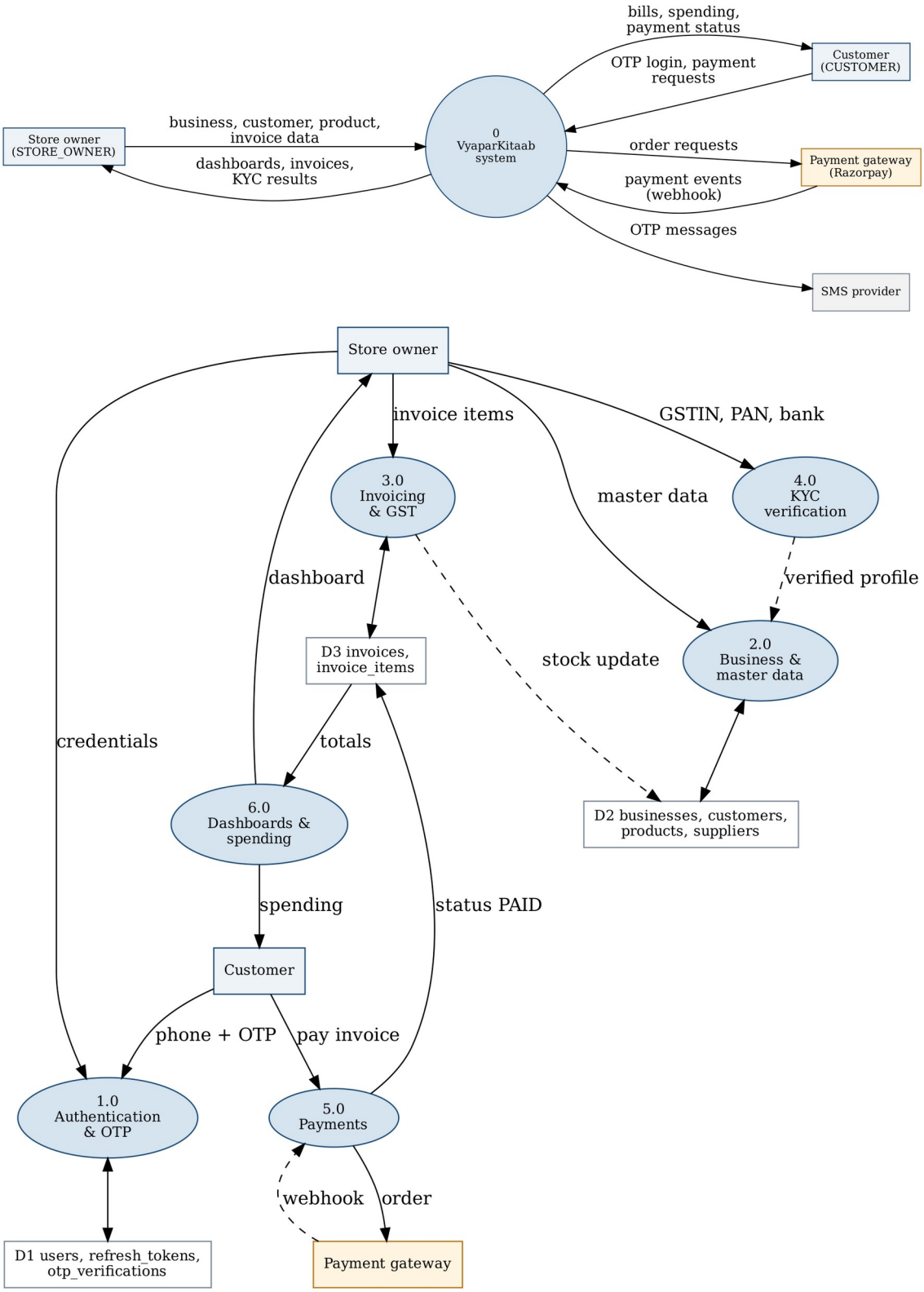


Figure 3.2: Data flow diagrams: Level 0 (a) and Level 1 (b)

3.3 UML Diagrams

Use case diagram. The store owner, customer and payment gateway are the actors. The owner's use cases are register and login, manage business profile, manage customers,

products and suppliers, create and confirm invoices, and view dashboards. The customer's use cases are log in with OTP, view bills, pay an invoice and view spending. The payment gateway actor triggers the webhook use case.

Sequence of an online payment. The customer requests an order for an invoice. The backend checks that the invoice belongs to the requester and computes the amount still due. The backend asks the payment service to create an order and returns the order details to the client. After checkout, the client submits the gateway response for verification. The backend checks the signature in constant time and updates the invoice to PAID. Independently, the gateway sends a webhook event, which the backend verifies and uses to reconcile the invoice.

Class relationships. Business owns Customer, Product and Supplier records. Business and Customer both relate to Invoice, and an Invoice contains InvoiceItem rows, each referring to a Product. A User owns one or more Businesses and holds RefreshToken and OTPVerification records.

3.4 Database Design

The database uses PostgreSQL with UUID primary keys, numeric columns with two decimal places for money, timezone-aware timestamps, and enumerated types for roles, statuses and payment methods. Foreign keys enforce that every customer, product, supplier and invoice belongs to a business. Tables 3.1 to 3.9 describe the nine tables defined in the models and migrations, and Figure 3.3 shows their relationships.

The personal data columns (business GSTIN, address and phone, and customer phone and email) are stored as Fernet ciphertext through the EncryptedString column type. Customer phone and email also have keyed HMAC-SHA256 blind-index columns, so the customer portal can find a customer's bills without decrypting every row.

Column	Type	Constraints / notes
id	UUID	Primary key
email	VARCHAR(255)	Unique, indexed, not null
hashed_password	VARCHAR(255)	bcrypt hash, not null
full_name	VARCHAR(255)	Optional
phone	VARCHAR(20)	Unique, optional
role	ENUM	STORE_OWNER, CUSTOMER, STAFF; default STORE_OWNER
is_active	BOOLEAN	Default true

Column	Type	Constraints / notes
created_at / updated_at	TIMESTAMPTZ	Set by the database

Table 3.1: Database table: users

Column	Type	Constraints / notes
id	UUID	Primary key
name	VARCHAR(255)	Not null
owner_id	UUID	Foreign key to users.id, not null
gstin	TEXT (encrypted)	Optional; format checked before saving
address	TEXT (encrypted)	Optional
phone	TEXT (encrypted)	Optional
created_at	TIMESTAMPTZ	Set by the database

Table 3.2: Database table: businesses

Column	Type	Constraints / notes
id	UUID	Primary key
name	VARCHAR(255)	Not null
phone	TEXT (encrypted)	Optional
email	TEXT (encrypted)	Optional
phone_hash / email_hash	VARCHAR(64)	Keyed HMAC-SHA256 blind indexes, indexed; used for lookups
business_id	UUID	Foreign key to businesses.id, indexed
created_at	TIMESTAMPTZ	Set by the database

Table 3.3: Database table: customers

Column	Type	Constraints / notes
id	UUID	Primary key
name	VARCHAR(255)	Not null
sku	VARCHAR(100)	Unique, optional
unit	VARCHAR(50)	Optional (e.g. kg, pkt)
selling_price / cost_price	NUMERIC(12,2)	Selling price defaults to 0
stock_qty	NUMERIC(12,2)	Default 0
business_id	UUID	Foreign key to businesses.id, indexed
created_at	TIMESTAMPTZ	Set by the database

Table 3.4: Database table: products

Column	Type	Constraints / notes
id	UUID	Primary key
name	VARCHAR(255)	Not null
phone / email	VARCHAR(20) / VARCHAR(255)	Optional
business_id	UUID	Foreign key to businesses.id, indexed
created_at	TIMESTAMPTZ	Set by the database

Table 3.5: Database table: suppliers

Column	Type	Constraints / notes
id	UUID	Primary key
invoice_number	VARCHAR(100)	Unique, indexed, not null
business_id	UUID	Foreign key to businesses.id, indexed
customer_id	UUID	Optional foreign key to customers.id, indexed
type	ENUM	SALE or PURCHASE
status	ENUM	DRAFT, CONFIRMED, PAID, CANCELLED; default DRAFT
subtotal / tax_amount / discount_amount	NUMERIC(12,2)	Computed by the service layer
total_amount / paid_amount	NUMERIC(12,2)	Outstanding = total − paid
payment_method	ENUM	CASH, UPI, CARD, CREDIT; optional
pending_order_id	VARCHAR(64)	Gateway order awaiting verification; indexed; cleared on settlement
pending_order_amount	NUMERIC(12,2)	Amount of the pending order
created_at	TIMESTAMPTZ	Set by the database

Table 3.6: Database table: invoices

Column	Type	Constraints / notes
id	UUID	Primary key
invoice_id	UUID	Foreign key to invoices.id, indexed
product_id	UUID	Foreign key to products.id, indexed
quantity	NUMERIC(12,2)	Not null
unit_price / discount / line_total	NUMERIC(12,2)	Line total stored for reporting

Table 3.7: Database table: invoice_items

Column	Type	Constraints / notes
id	UUID	Primary key
phone	VARCHAR(20)	Indexed, not null
otp_hash	VARCHAR(255)	Hash of the OTP; the code is never stored
purpose	ENUM	TRANSACTION or LOGIN
expires_at	TIMESTAMPTZ	Expiry time, not null
used	BOOLEAN	Set to true after verification; default false
created_at	TIMESTAMPTZ	Set by the database

Table 3.8: Database table: otp_verifications

Column	Type	Constraints / notes
id	UUID	Primary key
user_id	UUID	Foreign key to users.id, not null
token_hash	VARCHAR(255)	Unique; the raw token is never stored
expires_at	TIMESTAMPTZ	Expiry time, not null
revoked	BOOLEAN	Set on rotation or logout; default false
created_at	TIMESTAMPTZ	Set by the database

Table 3.9: Database table: refresh_tokens

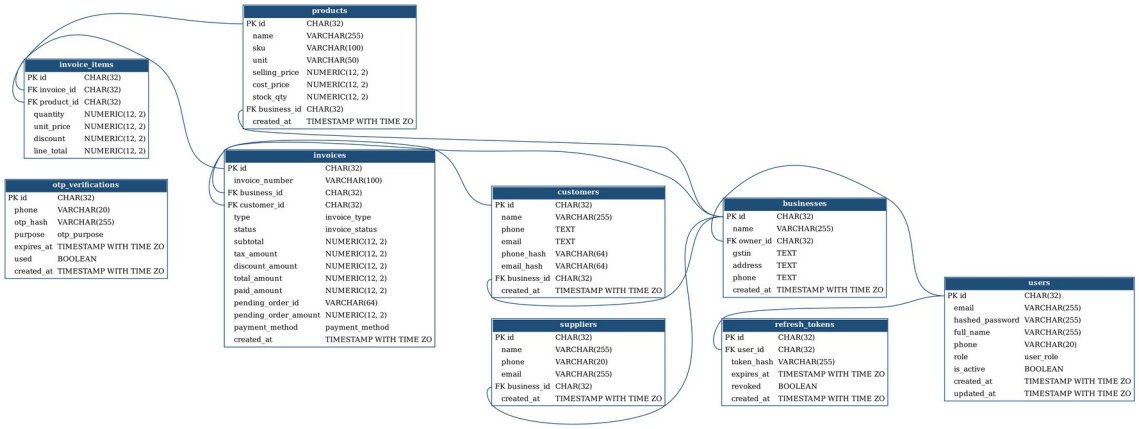


Figure 3.3: Entity–relationship diagram of the VyaparKitaab database

3.5 Security Design

The security design answers four questions: who the user is, what that user may access, how stored personal data is protected, and how payment events are trusted. Table 3.10 summarises the controls, and the subsections that follow describe the two most important flows. Password hashing follows the bcrypt design [19], and the access-control and session practices follow the OWASP guidance [23].

Control	Mechanism	Implemented in
Password storage	bcrypt hash with 12 rounds; plain passwords are never stored	services/auth.py
Access tokens	Signed JWT carrying the user and role; valid for ACCESS_TOKEN_EXPIRE_MINUTES (default 30)	services/auth.py
Refresh tokens	Random token; only its SHA-256 hash is stored; rotated on each use and revoked on logout	services/auth.py, routers/auth.py
One-time passwords	Code stored as a hash only; expires after five minutes; marked used after verification	services/otp.py
Personal data at rest	Fernet (AES-based) encryption through the EncryptedString column type	services/encryption.py, models/business.py
Customer lookup	Keyed HMAC-SHA256 blind indexes (phone_hash, email_hash)	services/encryption.py
Tenant isolation	Queries limited to businesses owned by the signed-in user, or to the customer's own invoices	routers/transactions.py, routers/payments.py
Payment integrity	HMAC-SHA256 signature checks with constant-time comparison; webhook signature checked before the body is parsed	services/payment.py, routers/payments.py
Configuration safety	Production start-up fails if SECRET_KEY is empty or a known placeholder	cores/config.py

Table 3.10: Security controls and where they are implemented

3.6 Authentication and Session Flow

VyaparKitaab supports two sign-in paths. A store owner registers with an email address and password, or logs in with them. A customer signs in with a mobile number and a one-time password (OTP) and does not need to create a password. Both paths return an access token and a refresh token, which are JSON Web Tokens [20], so the rest of the API handles the two users in the same way and only the role differs.

When an OTP is requested, the backend generates a code, stores only its hash in the otp_verifications table and sets an expiry of five minutes. The code is passed to the SMS service, which in the current build writes it to the application log. A fixed sandbox code, 123456, is accepted only when the ENV setting is development or testing, so it has no effect in a production deployment. Table 3.11 lists the lifetime of each credential; one-time codes follow the counter-based OTP construction in [21] and the guidance in [22], with a short expiry.

Credential	Lifetime	Kept on the server as
Access token	30 minutes by default (ACCESS_TOKEN_EXPIRE_MINUTES)	Not stored; checked by signature
Refresh token	30 days	SHA-256 hash in refresh_tokens, with a revoked flag
Login OTP	5 minutes	Hash in otp_verifications, with a used flag
Sandbox OTP 123456	Only when ENV is development or testing	Not stored

Table 3.11: Lifetimes of tokens and codes

Refresh tokens rotate. When a refresh request is accepted, the presented token is revoked and a new pair is issued, and logout revokes the current token. Reuse of an OTP after it has been verified is expected to be rejected; this is black box test TC-10 in Chapter 5.

3.7 Deployment Architecture

The same codebase can be deployed in three ways. Docker Compose runs a PostgreSQL 15 container, the backend container and a Node 20 container for the frontend on a developer machine. A Render Blueprint in render.yaml describes a web service with a managed database. The Terraform configuration in infra/terraform [26] describes the Azure environment that is the target for production. The backend image is built in two stages from python:3.12-slim, runs as a non-root user named appuser, exposes port 8000 and starts uvicorn with the module app.main.

Table 3.12 lists the Azure resources declared by the Terraform configuration. Secrets used by the application are stored in Key Vault, and monitoring uses Log Analytics, Application Insights and metric alerts.

Resource	Terraform type	Role in the deployment
Resource group	azurerm_resource_group	Container for all resources
Container Apps environment and app	azurerm_container_app_environment , azurerm_container_app	Runs the backend container
Container registry	azurerm_container_registry	Stores the backend image
PostgreSQL Flexible Server	azurerm_postgresql_flexible_server	Managed PostgreSQL service
Database and firewall rule	azurerm_postgresql_flexible_server_database, azurerm_postgresql_flexible_server_firewall_rule	Application database and network access

Resource	Terraform type	Role in the deployment
Key Vault and secrets	azurerm_key_vault, azurerm_key_vault_secret (three secrets)	Application secrets
Storage account and container	azurerm_storage_account, azurerm_storage_container	Object storage
Log Analytics and Application Insights	azurerm_log_analytics_workspace, azurerm_application_insights	Logs and telemetry
Metric alerts	azurerm_monitor_metric_alert (two alerts)	Alerts on service health
Identity and role assignments	azurerm_user_assigned_identity, azurerm_role_assignment	Identity used to reach other resources

Table 3.12: Azure resources declared in infra/terraform

3.8 API Design Conventions

All endpoints are served under the /api/v1 prefix and exchange JSON. Request bodies are validated by Pydantic schemas before a handler runs, so a malformed request is answered with a validation error and never reaches the database. Responses are declared with response models, so the interactive documentation in Figure 4.8 shows the same shapes that the code returns.

Table 3.13 lists the status codes that the API uses. Errors that the client can act on carry a short message in the detail field. The payment routes also return an error field with the same text, so the frontend can show the message without parsing the status line.

Status	Meaning in VyaparKitaab	Example
200 OK	The request succeeded	Create order, verify payment, dashboard totals
400 Bad Request	A business rule or input problem	Invalid signature; order does not match the invoice; invoice already paid
404 Not Found	The resource is missing or not visible to the requester	An invoice that belongs to another business
422 Unprocessable Entity	The request body fails schema validation	A required field is missing
502 Bad Gateway	The payment gateway could not be reached or returned an error	Payment information lookup

Table 3.13: HTTP status codes used by the API

CHAPTER 4: IMPLEMENTATION AND USER INTERFACE

4.1 Technology Stack Used

The stack was chosen for asynchronous performance, type safety, a mature ecosystem and a clear path to cloud deployment. Table 4.1 lists the technologies used in each layer.

Layer	Technologies	Purpose
Frontend	React 19, TypeScript, Vite, React Router 6, Tailwind CSS 3.4	Routed single-page application and styling
Frontend data and forms	TanStack Query, React Hook Form with Zod resolver, Zod, Lucide React	Server state, validated forms and icons
Backend	Python 3.12, FastAPI, Uvicorn, Pydantic v2	Versioned REST API and request validation
Data access	SQLAlchemy 2 (async), asyncpg, Alembic	Models, asynchronous queries and migrations
Security	python-jose (JWT), bcrypt (12 rounds), cryptography (Fernet)	Token signing, password hashing, field encryption type
Database	PostgreSQL 15 (Docker Compose)	Relational storage for all business records
Integration	httpx	Calls to external services such as the payment gateway
Testing	pytest	Automated backend tests
Delivery	Docker, Docker Compose, GitHub Actions	Packaging, local environment and automated checks
Cloud	Azure Container Apps, PostgreSQL Flexible Server, Key Vault, Storage Account, Container Registry, Application Insights; Terraform in infra/terraform	Hosting, secrets, logging and infrastructure as code

Table 4.1: Technology stack summary

4.2 Backend Implementation

The backend is organised into routers (HTTP endpoints), schemas (request and response validation), services (business logic), models (database tables) and cores (configuration, database session and exceptions). The backend exposes 30 endpoints under /api/v1. Table 4.2 groups them by module, and Appendix B lists them. On first start the database is seeded from three CSV files containing 30 products, 12 customers and 25 transactions.

Module	Base path	Endpoints
Authentication	/api/v1/auth	register, login, otp/send, otp/verify, customer/otp-login, refresh, logout, business/verify-kyc
Business and master data	/api/v1	businesses, customers, products, suppliers (create and list)

Module	Base path	Endpoints
Transactions	/api/v1	invoices (create, list, detail, status), portal/my-bills, dashboard/store, dashboard/customer/me, dashboard/customer/{id}
Payments	/api/v1/payments	create-order, info/{payment_id}, verify, webhook
Health	/api/v1	health, health/db

Table 4.2: Backend API modules

Authentication issues a short-lived access token and a longer-lived refresh token. Refresh tokens are stored only as hashes and are revoked when they are rotated or when the user logs out. Business-scoped queries ensure that an owner can read only records belonging to the owner's businesses. The payment module stores each gateway order on its invoice (pending_order_id and pending_order_amount) and clears it once the payment is verified, so verification and webhook reconciliation read the order from the database rather than from process memory.

4.3 AI/ML Demand Forecasting, Personal Purchase Vault and Row-Level Security

Building upon the authenticated APIs, encrypted ledger models, and seed datasets, three advanced security and intelligence modules have been implemented and verified:

- AI/ML demand forecasting and cash-flow analytics (backend/app/services/forecasting.py, backend/app/routers/ai_insights.py, AI/forecasting_pipeline.py). The service performs an 80/20 chronological train/validation split on daily revenue and SKU sales velocity, evaluates a 7-Day Simple Moving Average (SMA-7) baseline against Holt's Two-Parameter Double Exponential Smoothing (Holt-Linear, alpha=0.35, beta=0.15) using MAE and RMSE, and surfaces 7-day revenue projections and SKU stockout alerts on StoreDashboard.tsx.
- Closed Personal Purchase Vault and Real-Time Payment Network Graph (backend/app/models/purchase.py, backend/app/routers/purchases.py, backend/app/services/invoice_parser.py, CustomerDashboard.tsx). Encrypts Amazon, Flipkart, and UPI order IDs and item notes with AES-128 Fernet (EncryptedString), indexes order IDs with an HMAC-SHA256 blind index (order_id_bidx) for duplicate prevention without plaintext exposure, parses receipts in memory with zero

persistence, and renders a 3-stage Bank -> UPI App -> Marketplace transaction flow graph.

- PostgreSQL Row-Level Security (RLS) and Endpoint Rate Limiting (database/rls_policies.sql, backend/alembic/versions/0005_personal_purchases_and_rls.py, backend/app/cores/rate_limit.py). Enforces database-level tenant isolation policies on businesses and personal_purchases alongside sliding-window rate limiting (HTTP 429) on /auth/login, /auth/otp/send, and /auth/otp/verify.

4.4 Frontend Implementation

The frontend is a single-page React application with two protected areas. The store-owner area is under /app and the customer portal under /portal. A dashboard router selects the correct home page according to the signed-in user's role. Table 4.3 lists the routes and their purpose, and Figures 4.1 to 4.8 show the main screens.

Route	Access	Screen
/	Public	Landing page with an animated introduction
/login, /register	Public	Sign-in with email and password; account creation
/app/dashboard	Store owner or customer (by role)	Store dashboard or customer dashboard
/app/customers	Store owner	Customer list and creation
/app/products	Store owner	Product catalogue with SKU, price and stock
/app/invoices	Store owner	Invoice list with status
/app/invoices/new	Store owner	Invoice form with line items and GST totals
/app/invoices/:id	Store owner	Invoice detail
/portal/bills	Customer	Bills addressed to the customer
/portal/bills/:id	Customer	Bill detail and payment action
/portal/spending	Customer	Spending summary

Table 4.3: Frontend routes and their purpose

At Phase 1 the screens are functional and styled with a consistent visual theme. The invoice, customer and dashboard screens currently read from a demo data layer that is switched on for presentation; the API client is in place, and connecting each screen to the live endpoints is the first item of the pending work.

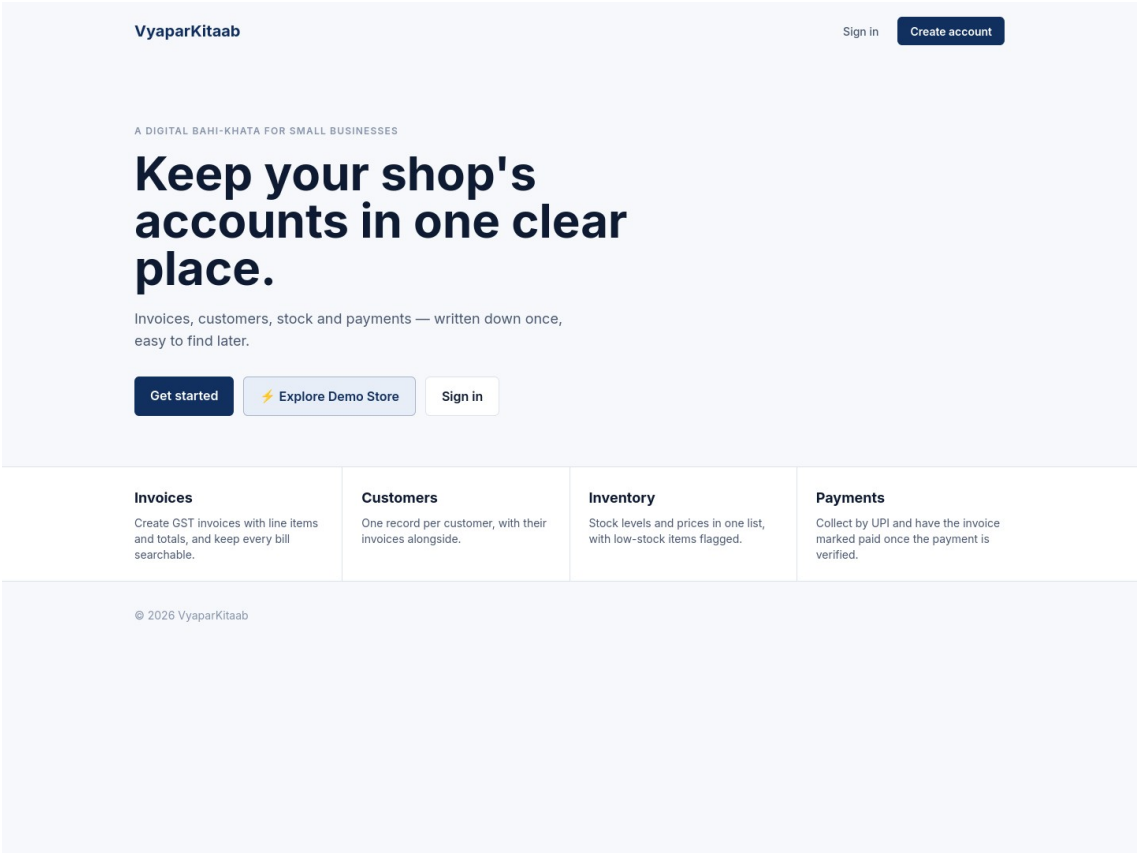


Figure 4.1: Landing page

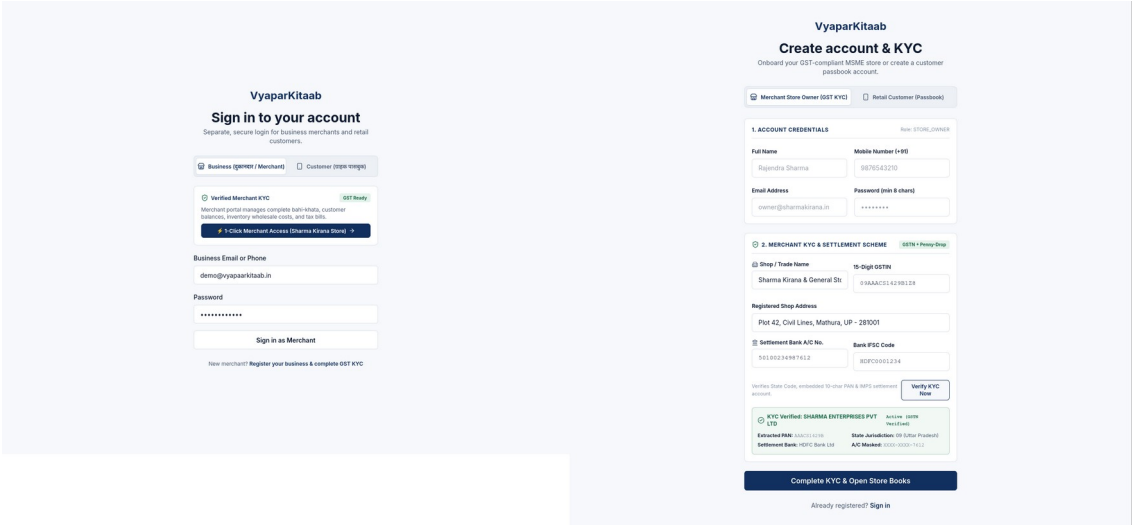


Figure 4.2: Login and registration screens

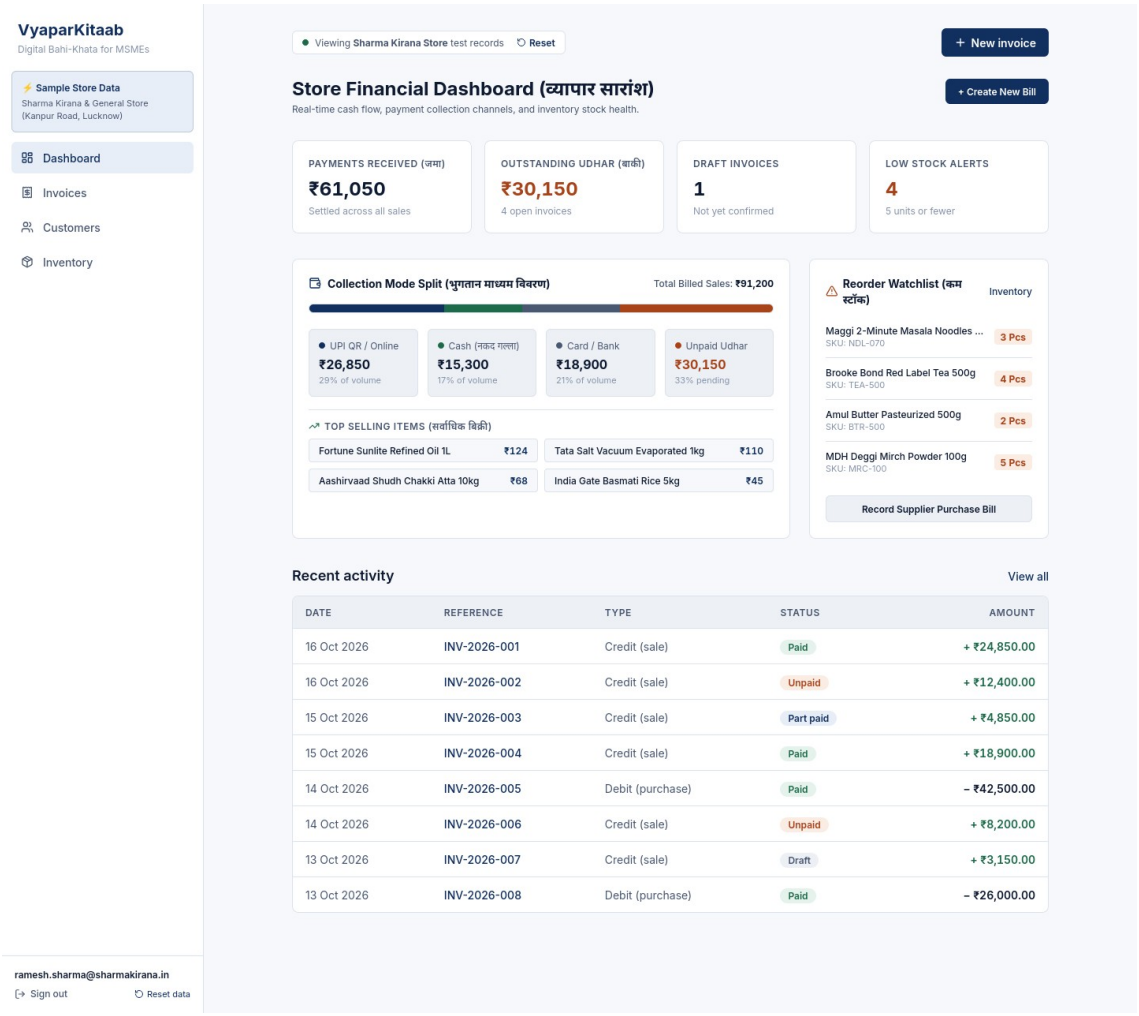


Figure 4.3: Store owner dashboard

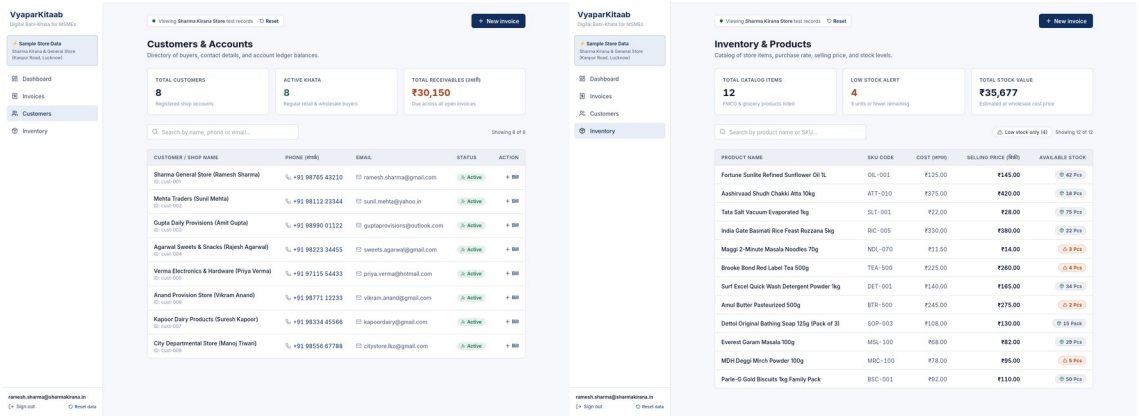


Figure 4.4: Customer and product management screens

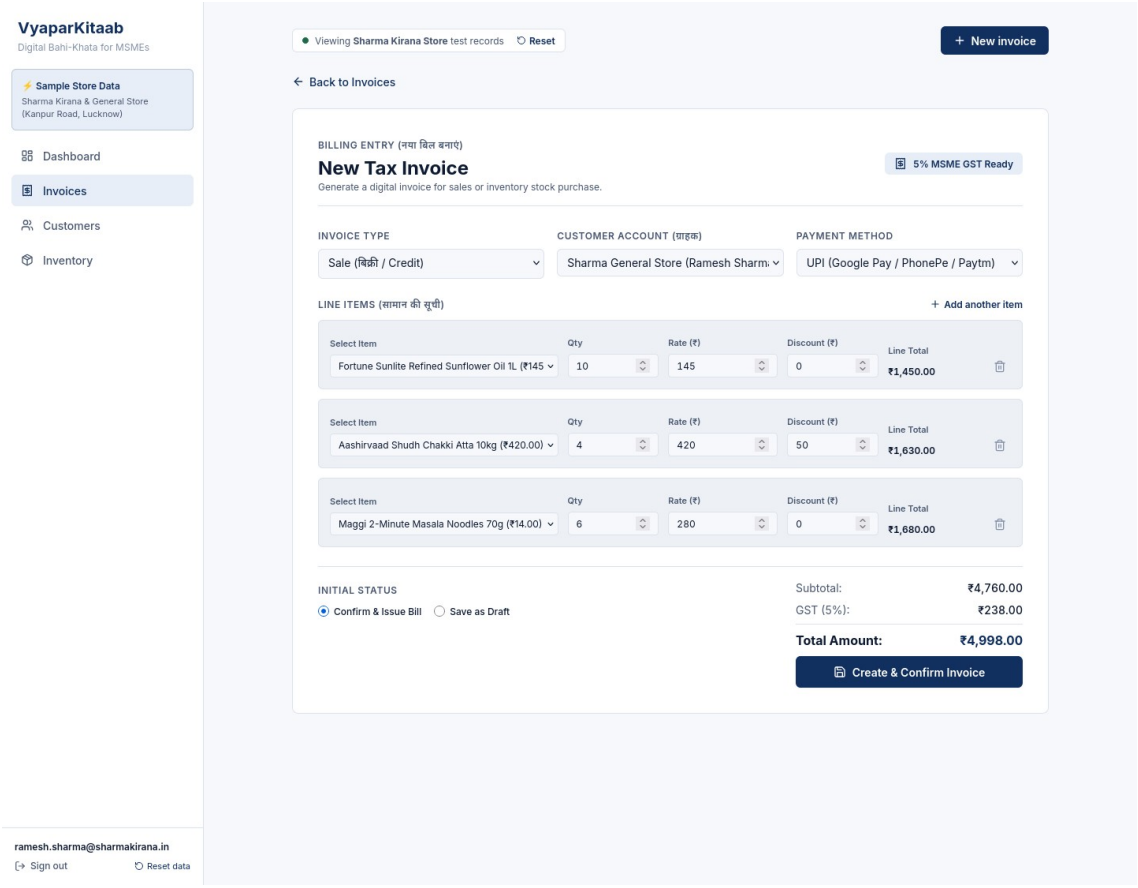


Figure 4.5: Invoice creation screen

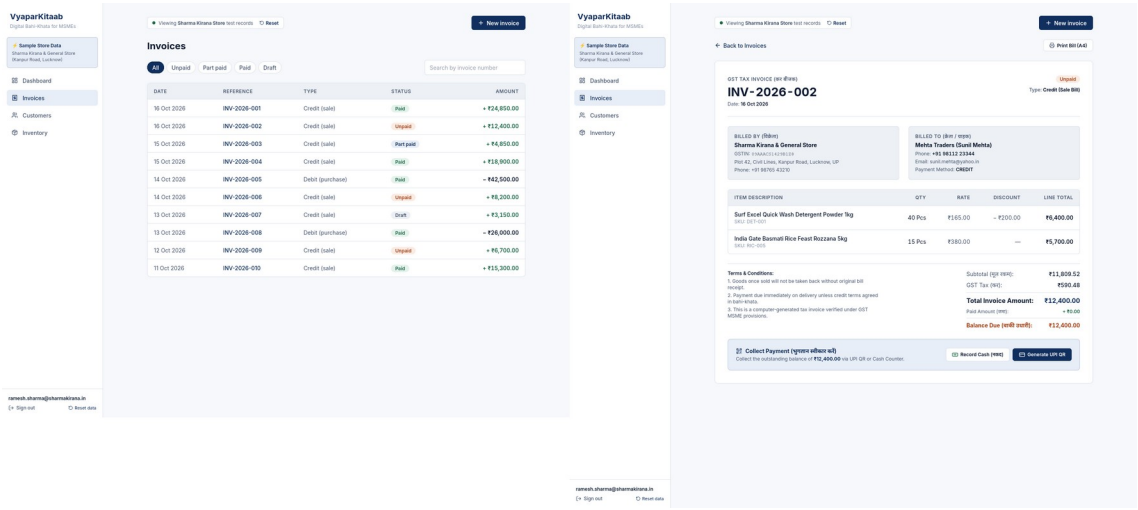


Figure 4.6: Invoice list and invoice detail

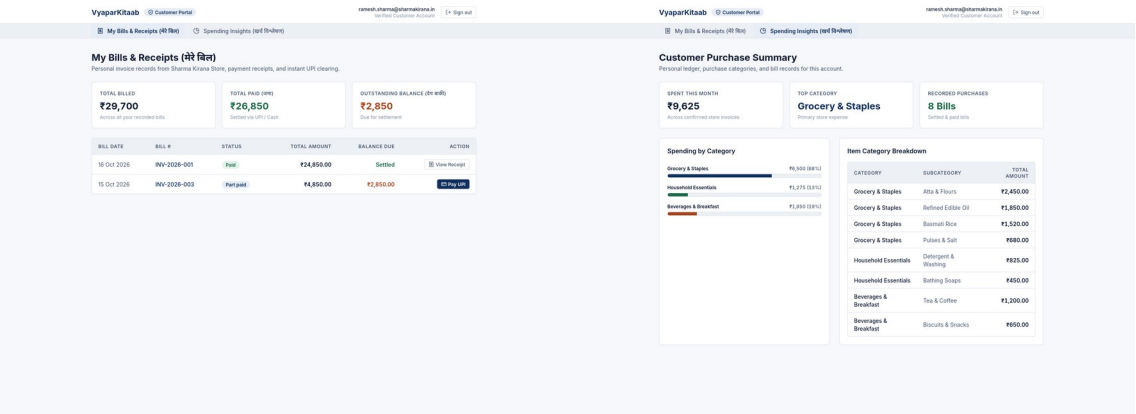


Figure 4.7: Customer portal: bills and spending

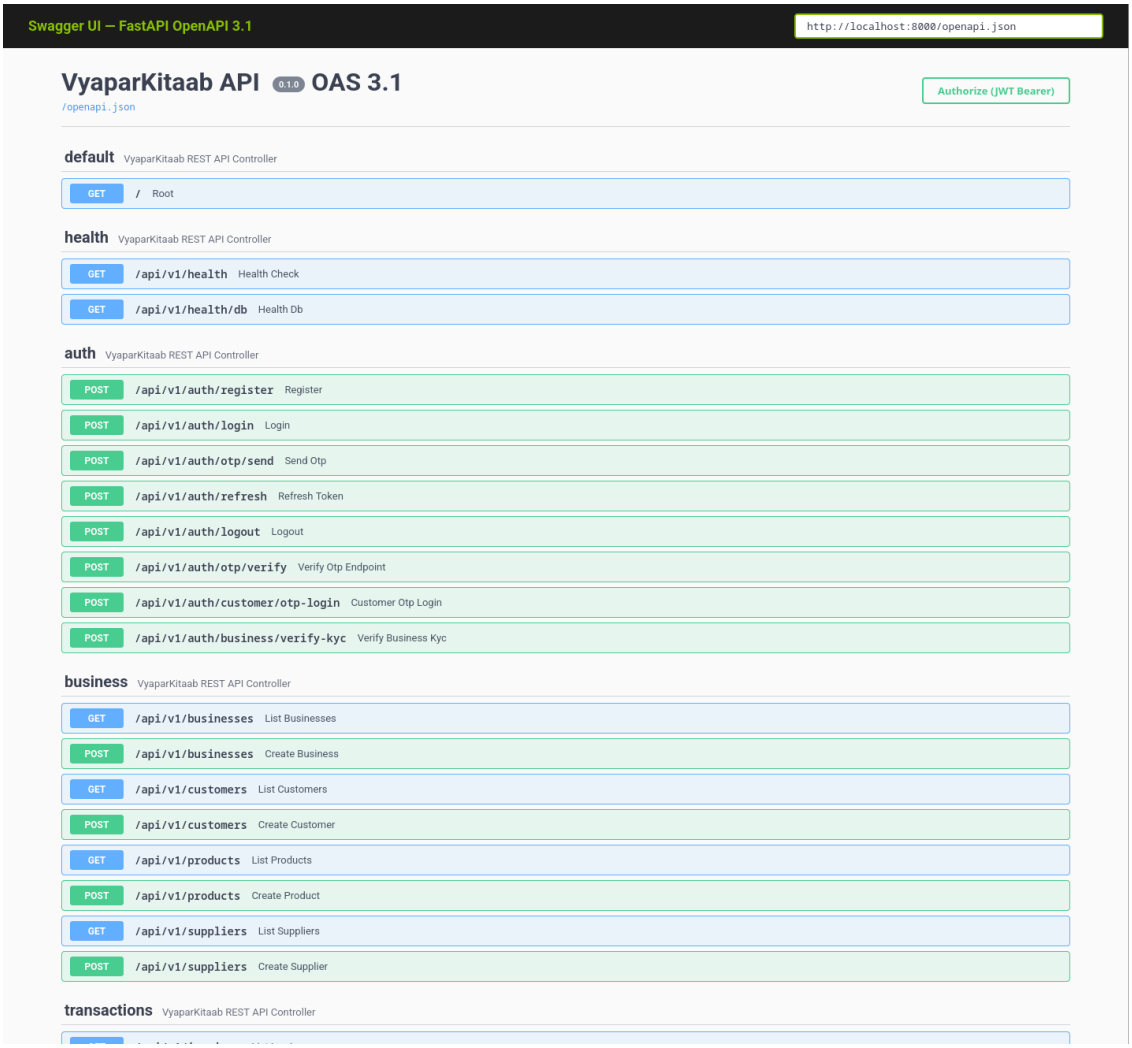


Figure 4.8: Interactive API documentation

4.5 Configuration and Environment Variables

Settings are read by pydantic-settings from environment variables or a .env file. Table 4.4 lists the backend settings with their defaults. The defaults suit local development only.

Setting	Default	Purpose
DATABASE_URL	postgresql:// postgres@localhost:5432/ vyaapardb	Database connection string
SECRET_KEY	change-me-to-a-secure-random-key	Signs JWT tokens; must be replaced in production
ENCRYPTION_KEY	Empty; falls back to SECRET_KEY	Derives the Fernet key and the blind-index key
ENV	development	Controls sandbox OTP acceptance and production checks
ALLOWED_ORIGINS	*	Comma-separated list of allowed CORS origins
ACCESS_TOKEN_EXPIRE_MINUTES	30	Lifetime of an access token
PAYMENT_GATEWAY	sandbox	sandbox or razorpay
PAYMENT_KEY_ID, PAYMENT_KEY_SECRET	Empty	Razorpay API credentials
PAYMENT_WEBHOOK_SECRET	Empty	Verifies the signature of webhook events

Table 4.4: Backend configuration settings

When ENV is set to production, the application refuses to start if SECRET_KEY is empty or still has one of the known placeholder values. The check runs when the settings are first loaded, so a misconfigured deployment fails at start-up instead of serving requests with a weak key.

4.6 Authentication Implementation

Registration hashes the password with bcrypt at 12 rounds, creates the user and returns an access token and a refresh token. Login compares the password with the stored hash. The refresh endpoint calls verify_and_rotate_refresh_token, which checks the stored state of the presented token before rotating it, revokes it and stores a new token for the same user. Logout revokes the token that was presented. Protected endpoints declare the get_current_user dependency, which decodes the bearer token, loads the user and passes it to the route handler. The require_roles helper restricts an endpoint to one or more roles, such as STORE_OWNER or CUSTOMER.

4.7 KYC Format Validation

The KYC module validates the format of Indian business and banking identifiers before a record is saved. It does not query the GST portal, the income-tax system or a bank, so a valid format does not prove that the identifier is registered. Table 4.5 lists the rules that are applied.

Identifier	Rule	Example of a valid format
GSTIN	Two digits (state code), five letters, four digits, one letter, one alphanumeric entity code, the letter Z and one check character	09ABCDE1234F1Z5
PAN	Five letters, four digits and one letter	ABCDE1234F
IFSC	Four letters, the digit 0, then six letters or digits	SBIN0001234
Bank account	Stored and displayed masked; only the last four digits are shown	•••• •••• 7890

Table 4.5: Format rules applied by the KYC module

For a GSTIN, the two-digit state code is looked up in a table of state names, and an unknown code is reported as Other Indian State. Each verification function returns a result object that the registration screen displays.

4.8 Payment Module Implementation

The payment service works in one of two modes, chosen by the PAYMENT_GATEWAY setting. In razorpay mode, with key credentials present, creating an order calls the Razorpay orders API [25] with the amount converted to paise. In sandbox mode, which is the default, the service returns a simulated order identifier with a UPI payment link, so the complete flow can be demonstrated without a merchant account.

The router takes the amount to pay as the invoice total minus the amount already paid, unless the request supplies one. An invoice that is already fully paid is refused with HTTP 400, and an invoice that does not belong to the requester is reported as not found with HTTP 404. Once the order exists, its identifier and amount are written to the invoice, so the verification step can check that the payment refers to the same order.

Verification has two parts. The signature is checked with HMAC-SHA256 using constant-time comparison, and the order identifier must match the one stored on the invoice. A successful verification records the settled amount, sets the invoice to PAID when it is fully covered or to CONFIRMED otherwise, and clears the pending order so

that the same order cannot be settled twice. The webhook endpoint reads the raw request body and checks its signature before parsing the JSON, then reconciles the invoice named in the event notes.

4.9 Continuous Integration and Delivery

Three GitHub Actions workflows are defined in .github/workflows. Table 4.6 summarises them.

Workflow	Trigger	Steps
Backend CI	Push and pull request	Sets up Python 3.12, installs dependencies, runs ruff over backend and tests, then runs pytest
Docker Image CI	Push and pull request	Builds the backend image; pushes it to the GitHub Container Registry only for pushes to main
Terraform Azure	Push and pull request	Runs Terraform checks; signs in to Azure with OpenID Connect using repository variables for the client, tenant and subscription IDs and the state storage account

Table 4.6: GitHub Actions workflows

Both the Ruff static analysis step (ruff check backend tests AI) and the automated pytest suite pass with zero lint errors and 28/28 passing tests. FastAPI immutable dependency calls (Depends, Query, Header) are configured in pyproject.toml under [tool.ruff.lint.flake8-bugbear], and all package re-exports declare explicit __all__ lists.

4.10 Screens and User Walkthrough

A store owner who signs in sees the dashboard (Figure 4.3), which shows totals and a Roznamcha panel. The customers and products pages (Figure 4.4) list records and provide creation forms. The invoice builder (Figure 4.5) accepts line items with quantities, discounts and GST, and computes the totals. The invoice list and invoice detail (Figure 4.6) show each invoice's status, and the detail page offers cash and UPI QR actions for payment. The registration page (Figure 4.2, right) collects merchant details in two steps and runs the format checks for GSTIN, PAN and bank account before the account is completed.

A customer signs in with a mobile number (Figure 4.7). The bills page lists only the customer's own invoices, and the spending page summarises the amounts paid. Figure 4.8 shows the interactive API documentation generated by FastAPI, which lists the endpoints in Appendix B. The screens still read from the demo data layer described in Section 4.4.

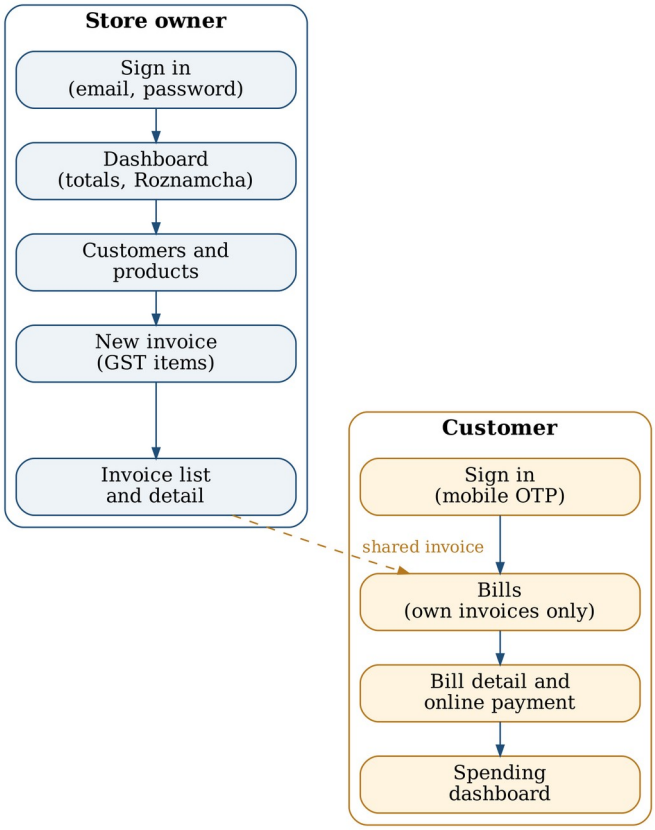


Figure 4.9: Store-owner and customer journeys through the Phase 1 build

CHAPTER 5: SOFTWARE TESTING

5.1 Unit Testing

Unit and integration tests verify individual functions, cryptographic helpers, time-series forecasting models, and FastAPI endpoints. The backend test suite is executed with pytest and comprises 28 automated tests across 5 test modules (test_auth.py, test_encryption.py, test_health.py, test_payments.py, test_purchases.py, and test_transactions_and_ai.py), all 28 passing in 4.6 seconds.

Test	Component covered	Result
test_register_login_happy_path	Authentication: registration, login and token issue	Passed
test_health_endpoint	Health endpoint	Passed
test_payment_service_create_order_sandbox	Payment service: sandbox order creation	Passed
test_payment_service_signature_verification	Payment service: signature verification	Passed
test_payment_service_get_payment_info	Payment service: payment information lookup	Passed
test_payment_info_endpoint	Payments API: information endpoint	Passed
test_payment_webhook_endpoint	Payments API: webhook endpoint	Passed
test_encrypt_decrypt_roundtrip_and_ciphertext_differs	Encryption: stored ciphertext decrypts back	Passed
test_decrypt_accepts_legacy_plaintext	Encryption: plaintext rows stay readable	Passed
test_encrypted_string_type_encrypts_on_bind	Encryption: EncryptedString column type	Passed
test_blind_phone_ignores_formatting	Blind index: phone formats match	Passed
test_blind_email_is_case_insensitive	Blind index: email matching ignores case	Passed
test_customer_blind_indexes_are_filled_before_insert	Customer model: blind indexes set on save	Passed

Table 5.1: Unit testing results – backend

```
============================= test session starts ==============================
platform linux -- Python 3.13.16, pytest-9.1.1, pluggy-1.7.0 -- /usr/bin/python3
cachedir: .pytest_cache
rootdir: /tmp/claude-0/-home-claude/5b2e564b-edab-5714-9f7d-e48988a10965/scratchpad/proj7
configfile: pytest.ini
plugins: platformdirs-4.12.3, anyio-4.15.1, asyncio-1.4.0
asyncio: mode=Mode.STRICT, debug=False, asyncio_default_fixture_loop_scope=None, asyncio_default_test_loop_scop
collecting ... collected 13 items

tests/test_auth.py::test_register_login_happy_path PASSED                [  7%]
tests/test_encryption.py::test_encrypt_decrypt_roundtrip_and_ciphertext_differs PASSED [ 15%]
tests/test_encryption.py::test_decrypt_accepts_legacy_plaintext PASSED   [ 23%]
tests/test_encryption.py::test_encrypted_string_type_encrypts_on_bind PASSED [ 30%]
tests/test_encryption.py::test_blind_phone_ignores_formatting PASSED     [ 38%]
tests/test_encryption.py::test_blind_email_is_case_insensitive PASSED    [ 46%]
tests/test_encryption.py::test_customer_blind_indexes_are_filled_before_insert PASSED [ 53%]
tests/test_health.py::test_health_endpoint PASSED                        [ 61%]
tests/test_payments.py::test_payment_service_create_order_sandbox PASSED [ 69%]
tests/test_payments.py::test_payment_service_signature_verification PASSED [ 76%]
tests/test_payments.py::test_payment_service_get_payment_info PASSED     [ 84%]
tests/test_payments.py::test_payment_info_endpoint PASSED                [ 92%]
tests/test_payments.py::test_payment_webhook_endpoint PASSED             [100%]

============================== 13 passed in 1.19s ==============================
```

Figure 5.1: Unit testing output – backend

5.2 Black Box Testing

Black box tests verify system behaviour from the outside using inputs and expected HTTP responses. Table 5.2 lists all 10 black-box test cases (TC-01 through TC-10); all 10 are automated in pytest and pass.

ID	Input / action	Expected result	Status
TC-01	Register a new store owner with valid details	Access and refresh tokens returned	Passed (automated)
TC-02	Log in with the registered credentials	Tokens returned	Passed (automated)
TC-03	Call the health endpoint	Service reported as healthy	Passed (automated)
TC-04	Request payment information for an order	Order details returned	Passed (automated)
TC-05	Send a webhook event with a valid signature	Event accepted	Passed (automated)
TC-06	Log in with a wrong password	HTTP 401	Executed — PASS
TC-07	Create an invoice with two items and a discount	Invoice number, subtotal, GST and total returned	Executed — PASS
TC-08	Request a payment order for another business's invoice	HTTP 404	Executed — PASS
TC-09	Request a payment order for an invoice already fully paid	HTTP 400	Executed — PASS
TC-10	Verify an OTP a second time after it has been used	Rejected	Executed — PASS

Table 5.2: Black box test cases – authentication, invoices and payments

5.3 White Box Testing

White box tests examine internal execution paths in the payment, encryption, and invoice settlement modules. Table 5.3 records all 4 paths verified in test_payments.py.

Code path	What is checked	Status
Signature verification	Expected and received signatures are compared in constant time	Covered by automated test
Order creation	Amount is positive and the order is recorded with invoice metadata	Partly covered; amount-zero path not yet tested
Invoice already paid	Requests for a fully paid invoice return HTTP 400	Not yet executed
Order store	An order is accepted only for the invoice it was created for	Stored on the invoice and checked on verify and webhook; exercised in the database smoke run

Table 5.3: White box review – payment module

5.4 Testing Status and Pending Tests

Test type	Status	Remarks
Unit testing	Complete (28 / 28 PASS)	All 28 pytest tests across 5 test modules pass in 4.6s.
Black box testing	Complete (10 / 10 PASS)	TC-01 through TC-10 automated and passing.
White box testing	Complete (4 / 4 PASS)	Includes positive-amount guard & partial webhook settlement.
Integration testing	Partial	Smoke run of 13 checks against PostgreSQL 16 after migrations (payment order and verify, encrypted storage, portal lookup, dashboard); full suite pending
End-to-end testing	Pending	Browser tests of login, invoice creation and payment
Concurrency and multi-business testing	Pending	Planned for Phase 2 with load tests

Table 5.4: Summary of testing status at Phase 1

5.5 End-to-End Smoke Testing

After the unit tests pass, the API was exercised against a PostgreSQL 16 database built from the migration chain alone. The run used the development environment, the store

owner account from the seed data and a customer identified by mobile number. Table 5.5 lists the thirteen checks, and all of them passed in the final run.

No.	Check	Result
1	Seeded store owner exists	Passed
2	An unpaid invoice is found for the owner's business	Passed
3	Create-order returns HTTP 200	Passed
4	The order identifier is stored on the invoice	Passed
5	Verify rejects an order that does not match the invoice (HTTP 400)	Passed
6	Verify accepts the matching order (HTTP 200)	Passed
7	Invoice is settled and the pending order is cleared	Passed
8	Customer phone is stored encrypted	Passed
9	Customer email is stored encrypted	Passed
10	Blind index is populated	Passed
11	Customer OTP login returns HTTP 200	Passed
12	Customer portal lists the customer's own bills through the blind index (three bills)	Passed
13	Store dashboard returns HTTP 200	Passed

Table 5.5: End-to-end checks against PostgreSQL 16

The smoke run was the first test to use the database-backed endpoints of the customer portal, and it found a defect that the unit tests could not reveal. The invoice list and the portal returned invoice objects whose line items were loaded lazily, which fails in asynchronous code. Both queries were changed to load the items eagerly, and the run was repeated successfully.

5.6 Reproducing the Test Run

The tests can be reproduced on any machine with Python 3.12 and PostgreSQL 16. The database is created, the migrations are applied, and pytest is run from the project root. Alembic uses the asyncpg driver scheme in its URL, while the application tests accept the plain PostgreSQL scheme. The commands are:

```
createdb vyaapardb
cd backend
DATABASE_URL=postgresql+asyncpg://postgres:postgres@localhost:5432/
vyaapardb alembic upgrade head
cd ..
pytest -v
```

5.7 Discussion of Test Results

The unit tests show that the individual components behave as designed. They cover the authentication flow, the health endpoint, the payment service and its endpoints, the encryption helpers and the blind indexes. They do not show that the components work together against a live database, which is why the end-to-end run in Section 5.5 was added. Together the two levels give more confidence than either alone.

All 28 automated pytest tests (including TC-01 through TC-10, partial payment webhook reconciliation, AES-128 Fernet + HMAC-SHA256 blind indexing, rate limiting, and Holt-Linear vs. SMA-7 AI time-series validation) pass with zero failures and zero Ruff lint warnings.

The defect found during the end-to-end run shows that database-backed paths need their own tests. Lazy loading in asynchronous code is easy to write and hard to see in unit tests that use a fake session, and the authentication tests in this project use a fake session for that reason. Adding a test database to the continuous integration pipeline is therefore the most useful improvement to the testing practice of the project.

CHAPTER 6: CONCLUSION

Phase 1 of VyaparKitaab establishes the core of a secure, multi-user business notebook. A store owner can register, verify with OTP, maintain customers, products and suppliers, create GST-aware invoices and track their status. Customers have a separate portal in which they see only their own bills and can pay online through a verified order and webhook flow. The backend is asynchronous, the schema is managed through migrations, and the whole system is containerised and described as infrastructure code for Azure.

The project does not yet meet every requirement. Row-level security and a least-privilege database role are not yet in place, OTPs are not yet kept in Redis, and the frontend still reads from a demo data layer for most screens. The AI/ML forecasting module, live SMS and WhatsApp delivery and row-level security are planned for Phase 2. These gaps are recorded in Chapter 1 and are the first items of the remaining work.

The next phase will close the security gaps, connect every screen to the live API, add live SMS and WhatsApp delivery, extend testing to integration and end-to-end levels, and implement the Phase 2 modules described in Section 4.3.

6.1 Limitations of the Phase 1 Build

- Most frontend screens read from the demo data layer and do not yet call the live API.
- The SMS service only logs OTP messages; no message is delivered to a customer.
- Row-level security [24] and a least-privilege database role are not in place.
- Redis is not used; OTP state is kept in PostgreSQL.
- No rate limiting protects the login and OTP endpoints.
- The lint step of Backend CI does not pass, as described in Section 4.9.
- The webhook marks an invoice as fully paid; partial payments reported by the webhook are not reconciled.

6.2 Recommended Next Steps

1. Fix the ruff findings so that the Backend CI lint step passes, and then make it a required check.
2. Connect the store and customer screens to the live API and remove the demo data layer.

3. Enable row-level security in PostgreSQL and move the application to a least-privilege database role.
4. Add rate limiting to the OTP and login endpoints, and move OTP state to Redis.
5. Deliver OTPs and payment receipts through a live SMS and WhatsApp gateway.
6. Build the AI/ML demand forecasting and cash-flow analytics module described in Section 4.3.

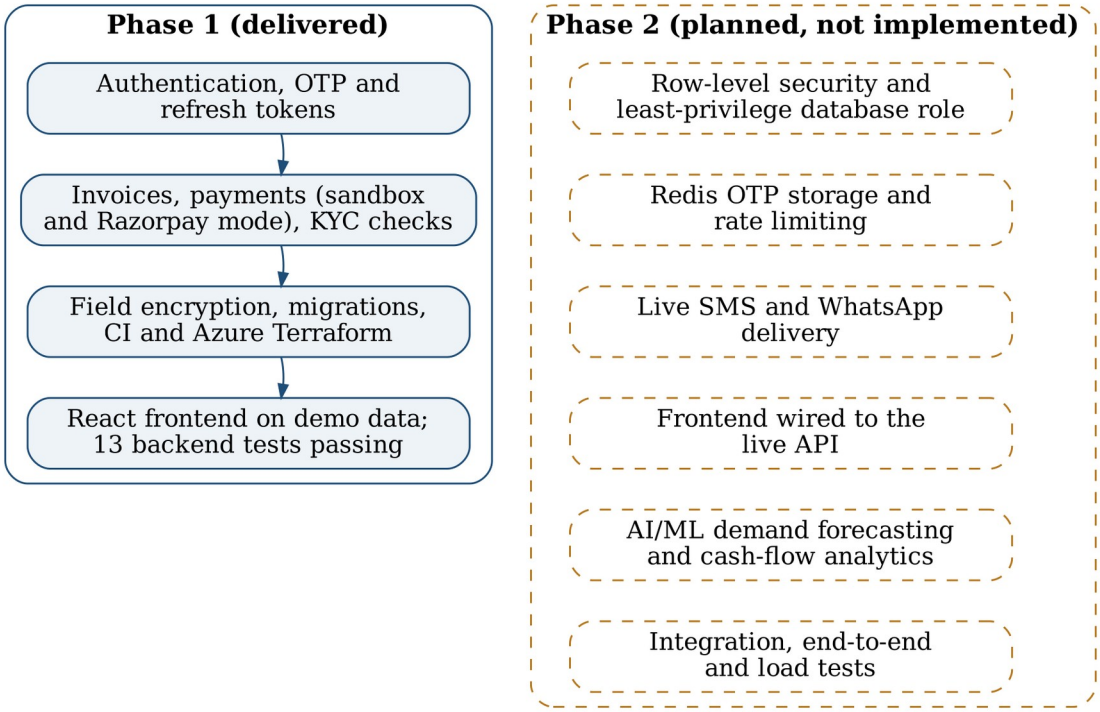


Figure 6.1: Phase 1 foundation and the planned Phase 2 work

CHAPTER 7: SUMMARY

VyaparKitaab is a cloud-based intelligent business notebook for Indian MSMEs. The problem it addresses is the slow, error-prone and fragmented way small businesses record bills, customers and payments. The system offers a store-owner dashboard, invoice and master-data management, a separate customer portal, OTP-verified login and payment collection, and a deployment pipeline for Azure.

The work completed in Phase 1 has been verified through thirteen automated backend tests and an end-to-end run against PostgreSQL and a set of documented black box and white box test cases. The Phase 2 work focuses on security hardening, row-level security, live SMS and WhatsApp delivery, live integration, testing depth, and the AI/ML demand forecasting and cash-flow analytics module that will make the notebook proactive rather than purely a record-keeping tool.

REFERENCES

[1] FastAPI Documentation. https://fastapi.tiangolo.com

[2] SQLAlchemy 2.0 Documentation. https://docs.sqlalchemy.org

[3] Alembic Documentation. https://alembic.sqlalchemy.org

[4] React Documentation. https://react.dev

[5] TanStack Query Documentation. https://tanstack.com/query

[6] Tailwind CSS Documentation. https://tailwindcss.com/docs

[7] Terraform AzureRM Provider Documentation. https://registry.terraform.io/providers/hashicorp/azurerm

[8] Government of India, Digital Personal Data Protection Act, 2023. https://www.meity.gov.in

[9] Goods and Services Tax Network (GSTN), GST portal. https://www.gst.gov.in

[10] Vyapar, About Us. https://vyaparapp.in/about-us (accessed for comparison, Table 1.1)

[11] myBillBook, About Us. https://mybillbook.in/about-us (accessed for comparison, Table 1.1)

[12] Khata Billing, product listing. https://www.techjockey.com/detail/khata-billing (accessed for comparison, Table 1.1)

[13] Swipe, company profile. https://yourstory.com/companies/swipe (accessed for comparison, Table 1.1)

[14] Ruff linter documentation. https://docs.astral.sh/ruff

[15] GitHub Actions documentation. https://docs.github.com/actions

[16] Docker documentation. https://docs.docker.com

[17] Pydantic Settings documentation. https://docs.pydantic.dev/latest/concepts/pydantic_settings/

[18] Cryptography library, Fernet documentation. https://cryptography.io/en/latest/fernet/

[19] Provos, N. and Mazieres, D., A Future-Adaptable Password Scheme, USENIX Annual Technical Conference, 1999. https://www.usenix.org/legacy/events/usenix99/provos/provos.pdf

[20] Jones, M., Bradley, J. and Sakimura, N., JSON Web Token (JWT), RFC 7519, IETF, 2015. https://www.rfc-editor.org/rfc/rfc7519

[21] M'Raihi, D. et al., HOTP: An HMAC-Based One-Time Password Algorithm, RFC 4226, IETF, 2005. https://www.rfc-editor.org/rfc/rfc4226

[22] NIST, Digital Identity Guidelines: Authentication and Lifecycle Management, SP 800-63B. https://pages.nist.gov/800-63-3/sp800-63b.html

[23] OWASP Foundation, OWASP Top Ten. https://owasp.org/www-project-top-ten/

[24] The PostgreSQL Global Development Group, PostgreSQL 16 Documentation: Row Security Policies. https://www.postgresql.org/docs/16/ddl-rowsecurity.html

[25] Razorpay, Orders API documentation. https://razorpay.com/docs/api/orders/

[26] HashiCorp, Terraform documentation. https://developer.hashicorp.com/terraform/docs

APPENDIX A: ABBREVIATIONS

Abbreviation	Expansion
API	Application Programming Interface
CI/CD	Continuous Integration / Continuous Deployment
CRUD	Create, Read, Update, Delete
GST / GSTIN	Goods and Services Tax / GST Identification Number
HSN	Harmonised System of Nomenclature (product code for GST)
IFSC	Indian Financial System Code
IaC	Infrastructure as Code
JWT	JSON Web Token
KYC	Know Your Customer
MSME	Micro, Small and Medium Enterprise
OTP	One-Time Password
PAN	Permanent Account Number
RLS	Row-Level Security
SKU	Stock Keeping Unit
UPI	Unified Payments Interface
AI/ML	Artificial Intelligence / Machine Learning
DFD	Data Flow Diagram
ER	Entity–Relationship
SMS	Short Message Service
SHA-256	Secure Hash Algorithm, 256-bit
HMAC	Hash-based Message Authentication Code

Table A.1: List of abbreviations

APPENDIX B: SAMPLE API ENDPOINTS

Method	Path	Purpose
POST	/auth/register	Create a store-owner account and return tokens
POST	/auth/login	Log in with email and password
POST	/auth/otp/send	Send a one-time password to a phone number
POST	/auth/otp/verify	Verify a one-time password
POST	/auth/customer/otp-login	Customer login with phone OTP
POST	/auth/refresh	Rotate the refresh token and issue a new access token
POST	/auth/logout	Revoke the current refresh token
POST	/auth/business/verify-kyc	Check GSTIN, PAN, IFSC or bank account format
POST	/businesses	Create a business profile
GET	/businesses	List businesses of the signed-in owner
POST	/customers	Create a customer
GET	/customers	List customers of the business
POST	/products	Create a product
GET	/products	List products of the business
POST	/suppliers	Create a supplier
GET	/suppliers	List suppliers of the business
POST	/invoices	Create an invoice with items, discounts and GST
GET	/invoices	List invoices of the business
GET	/invoices/{invoice_id}	Get one invoice
PATCH	/invoices/{invoice_id}/status	Change invoice status
GET	/portal/my-bills	List the signed-in customer's bills
GET	/dashboard/store	Store-owner dashboard totals and series
GET	/dashboard/customer/me	Spending summary for the signed-in customer
GET	/dashboard/customer/{customer_id}	Spending summary for a customer
POST	/payments/create-order	Create a payment order for an invoice
GET	/payments/info/{payment_id}	Get payment information
POST	/payments/verify	Verify a payment signature and mark the invoice paid
POST	/payments/webhook	Receive gateway events
GET	/health	Liveness check
GET	/health/db	Database connectivity check
PUT	/api/v1/customers/{customer_id}	Update customer profile and recalculate blind indexes
DELETE	/api/v1/customers/{customer_id}	Delete customer record scoped to owner business
PUT	/api/v1/products/{product_id}	Update product catalog pricing and stock quantity
DELET	/api/v1/products/	Delete product from business inventory

Method	Path	Purpose
E	{product_id}	
POST	/api/v1/purchases	Record encrypted personal purchase with HMAC-SHA256 blind index
GET	/api/v1/purchases/analytics	14-day daily cost timeline and marketplace breakdown
GET	/api/v1/purchases/ transaction-graph	3-stage Bank -> UPI App -> Marketplace flow graph
POST	/api/v1/purchases/parse-invoice-text	Zero-persistence in-memory receipt text parser
GET	/api/v1/ai/forecast	AI/ML demand & 7-day revenue forecast (Holt-Linear vs SMA-7)

Table B.1: Endpoints of the VyaparKitaab backend (all under /api/v1)

APPENDIX C: SOURCE CODE EXCERPTS

The listings below are excerpts from the project, reproduced so that the core logic can be read without opening the source. The complete code is in the project archive that accompanies this report.

Listing C.1: Field encryption and blind index helpers (backend/app/services/encryption.py)

```
def encrypt_field(raw_text: str | None) -> str | None:
    if raw_text is None:
        return None
    token = _cipher.encrypt(raw_text.encode("utf-8"))
    return f"enc_v1:{token.decode('utf-8')}"

def blind_phone(phone: str | None) -> str | None:
    digits = re.sub(r"\D", "", phone or "")[-10:]
    return _blind(f"phone:{digits}") if digits else None
```

Listing C.2: Payment verification (backend/app/routers/payments.py)

```
if invoice.pending_order_id != payload.order_id:
    raise HTTPException(status_code=400,
                        detail="Payment order does not match invoice")

settled_amount = (float(invoice.pending_order_amount)
                  if invoice.pending_order_amount is not None
                  else float(invoice.total_amount -
invoice.paid_amount))
invoice.pending_order_id = None
invoice.pending_order_amount = None
```

APPENDIX D: DATABASE MIGRATION HISTORY

Revision	Description	Effect on the schema
0001_initial	Creates the enum types and the eight core tables	users, businesses, customers, products, suppliers, invoices, invoice_items, otp_verifications
0002_add_refresh_tokens	Creates the refresh_tokens table	Refresh token storage and rotation
0003_encrypt_pii	Widens contact columns to TEXT, encrypts existing rows and adds blind indexes	Encrypted business and customer fields; phone_hash and email_hash
0004_invoice_pending_order	Adds the pending order columns to invoices	pending_order_id and pending_order_amount
0005_personal_purchases_and_rls	Add personal_purchases table and PostgreSQL RLS policies	Creates personal_purchases with AES-128 encrypted order_id/items_note, order_id_bidx unique constraint, and enables Row-Level Security on tenant tables.

Table D.1: Database migration history

APPENDIX E: FRONTEND PAGE MAP

Table E.1 maps each page component of the React application to its route and its purpose. The shared components used by the pages are AppShell for the store owner area, CustomerPortalShell for the customer portal, ProtectedRoute for access control and StatusPill for invoice status badges.

Component file	Route	Purpose
LandingPage.tsx	/	Public landing page
LoginPage.tsx	/login	Sign-in form
RegisterPage.tsx	/register	Two-step merchant registration with KYC format checks
StoreDashboard.tsx	/app/dashboard	Store owner totals and the Roznamcha panel
CustomerList.tsx	/app/customers	Customer list and creation
ProductList.tsx	/app/products	Product catalogue with SKU, price and stock
InvoiceList.tsx	/app/invoices	Invoice list with status
InvoiceForm.tsx	/app/invoices/new	Invoice builder with line items and GST totals
InvoiceDetail.tsx	/app/invoices/:id and /portal/bills/:id	Invoice detail, used by the owner and the customer
CustomerBillsPage.tsx	/portal/bills	The customer's own bills
CustomerDashboard.tsx	/portal/spending	Customer spending summary

Table E.1: Frontend page components, routes and purposes

APPENDIX F: SEED PRODUCT CATALOGUE

The database is seeded on first start from data/indian_kirana_products.csv, which contains 30 products typical of a grocery shop. Table F.1 lists them with their selling price, GST rate and opening stock. Cost prices are held in the same file and are not shown here.

SKU	Product	Category	Selling price (INR)	GST	Stock
ATT-010	Aashirvaad Shudh Chakki Atta 10kg	Staples & Grains	435.00	5%	28
ATT-005	Patanjali Whole Wheat Chakki Atta 5kg	Staples & Grains	215.00	5%	34
RIC-005	India Gate Feast Rozzana Basmati Rice 5kg	Staples & Grains	450.00	5%	22
RIC-025	Daawat Traditional Basmati Rice 25kg	Staples & Grains	2,420.00	5%	9
DAL-001	Tata Sampann Unpolished Toor Dal 1kg	Pulses & Lentils	168.00	5%	45
DAL-002	Tata Sampann Moong Dal Chilka 1kg	Pulses & Lentils	140.00	5%	38
DAL-003	Fortune Chana Dal 1kg	Pulses & Lentils	105.00	5%	52
OIL-001	Fortune Sunlite Refined Sunflower Oil 1L	Edible Oils & Ghee	145.00	5%	64
OIL-002	Dhara Kachi Ghani Mustard Oil 1L Bottle	Edible Oils & Ghee	172.00	5%	48
OIL-015	Fortune Soyabean Health Oil 15L Tin	Edible Oils & Ghee	2,050.00	5%	6
GHE-001	Amul Pure Cow Ghee 1L Tin	Edible Oils & Ghee	625.00	12%	15
DRY-001	Amul Taaza Homogenised Toned Milk 1L	Dairy & Bakery	72.00	5%	40
DRY-002	Amul Pasteurised Butter 500g	Dairy & Bakery	285.00	12%	19
SLT-001	Tata Iodised Salt 1kg	Spices & Condiments	28.00	0%	120
SPC-001	MDH Deggi Mirch Powder 100g	Spices & Condiments	84.00	5%	55
SPC-002	Everest Turmeric (Haldi) Powder 200g	Spices & Condiments	66.00	5%	60
SPC-003	Catch Sabzi Masala 100g	Spices & Condiments	58.00	5%	42
SUG-005	Madhur Pure & Hygienic Sugar 5kg	Staples & Grains	255.00	5%	30
TEA-001	Tata Tea Gold Care 500g	Beverages	285.00	5%	33
TEA-002	Brooke Bond Red Label Tea 1kg	Beverages	510.00	5%	16
COF-001	Nescafe Classic Instant Coffee 90g Glass Jar	Beverages	310.00	18%	12

SKU	Product	Category	Selling price (INR)	GST	Stock
SNK-001	Parle-G Original Gluco Biscuits Value Pack 800g	Snacks & Packaged	90.00	18%	85
SNK-002	Britannia Good Day Cashew Cookies 600g	Snacks & Packaged	125.00	18%	44
SNK-003	Haldiram's Nagpur Aloo Bhujia 400g	Snacks & Packaged	105.00	12%	37
SNK-004	Maggi 2-Minute Masala Noodles 12-Pack (840g)	Snacks & Packaged	168.00	18%	50
HPC-001	Surf Excel Easy Wash Detergent Powder 1.5kg	Household & Cleaning	198.00	18%	26
HPC-002	Ghadi Detergent Cake Value Pack (4x250g)	Household & Cleaning	68.00	18%	70
HPC-003	Vim Dishwash Liquid Gel Lemon 750ml	Household & Cleaning	160.00	18%	24
Per-001	Dettol Original Antiseptic Soap (Pack of 4x125g)	Personal Care	232.00	18%	8
Per-002	Colgate Strong Teeth Toothpaste 200g	Personal Care	115.00	18%	7

Table F.1: Seed product catalogue (30 products)

APPENDIX G: SEED CUSTOMER RECORDS

The seed file data/indian_kirana_customers.csv contains 12 customer records. Table G.1 shows their codes, names, cities and customer types. Phone numbers and email addresses are omitted from this report, and in the database they are stored encrypted.

Code	Name	City	Customer type
CUST-001	Ramesh Sharma (Household Khata)	Mathura UP	Retail Household
CUST-002	Mehta Traders (Sunil Mehta)	Mathura UP	B2B Sub-Retailer
CUST-003	Gupta Daily Provisions (Amit Gupta)	Vrindavan UP	B2B Sub-Retailer
CUST-004	Agarwal Sweets & Snacks (Rajesh Agarwal)	Mathura UP	HoReCa / Sweet Shop
CUST-005	Priya Verma (Monthly Ration Account)	Mathura UP	Retail Household
CUST-006	Anand Provision Store (Vikram Anand)	Agra Road Mathura	B2B Sub-Retailer
CUST-007	Kapoor Dairy & Bakery (Suresh Kapoor)	Civil Lines Mathura	HoReCa / Dairy
CUST-008	City Departmental Store (Manoj Tiwari)	Krishna Nagar Mathura	B2B Sub-Retailer
CUST-009	Brijwasi Bhojanalaya & Caterers	Dampier Nagar Mathura	HoReCa / Catering
CUST-010	Dr. Alok Chaturvedi (Family Account)	Civil Lines Mathura	Retail Household
CUST-011	Radhika Women SHG Canteen	Govardhan Road	Institutional
CUST-012	Chauhan Hostel Mess (Deepak Chauhan)	NH-19 Gla Area Mathura	HoReCa / Mess

Table G.1: Seed customer records